同济大学
TONGJI UNIVERSITY

同济大学
TONGJI UNIVERSITY

图像分析与机器学习

3 非线性分类/回归与多层感知机



张绍明



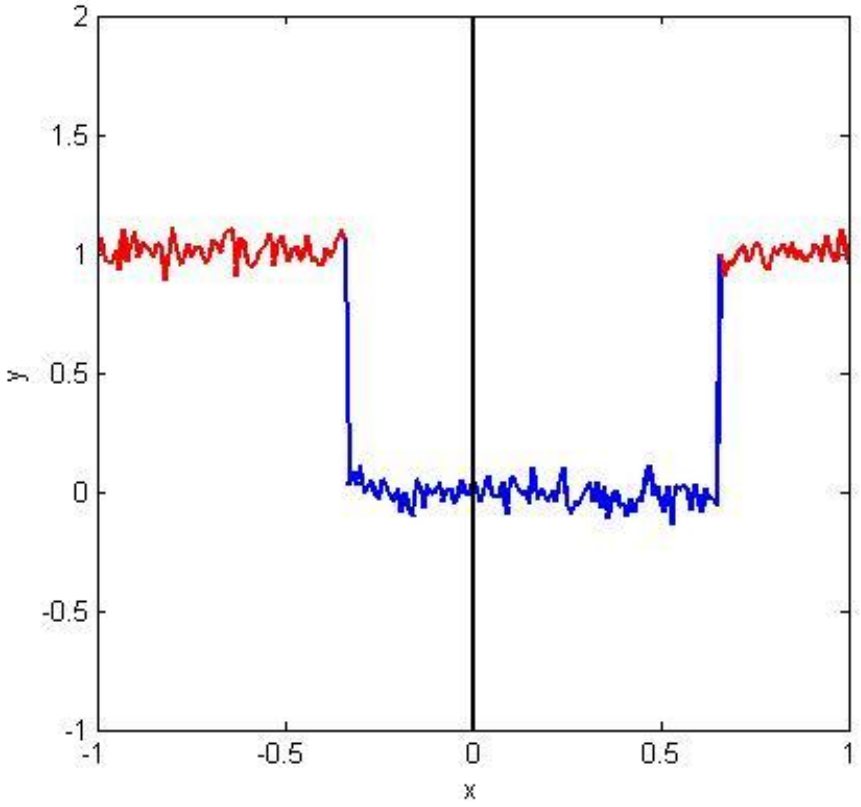
目录

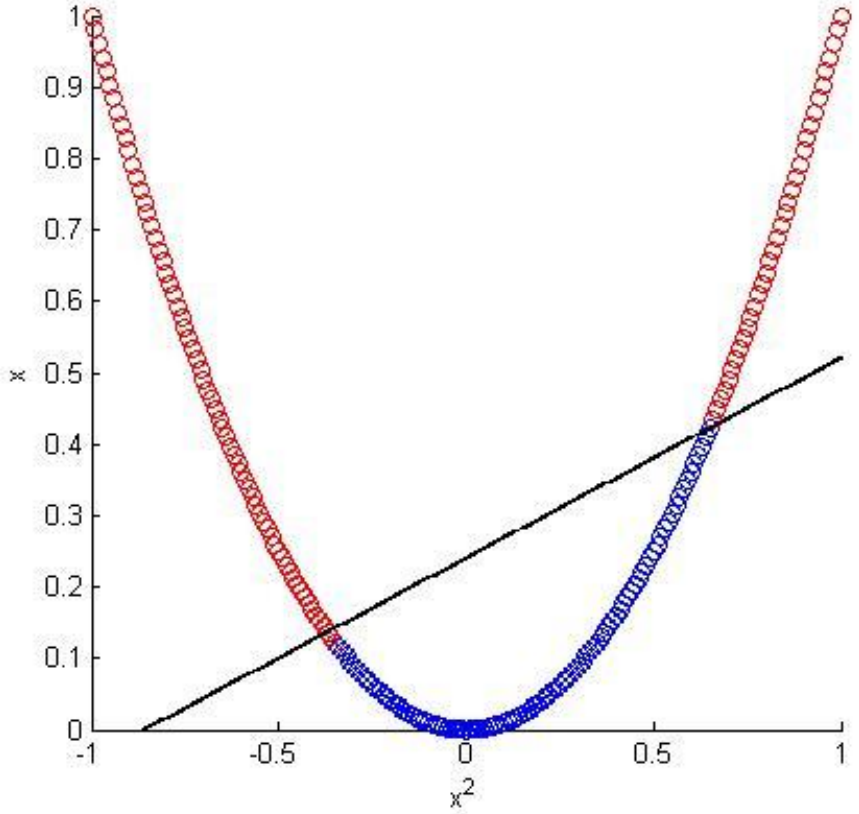
1. 神经网络的出发点
2. 前馈神经网络函数
3. 神经网络的训练
4. 后向传播算法

1 神经网络的出发点：维数灾难

1. 玄学？生物学？很多介绍都有博人眼球和言过其实之嫌！工程数学而已！
2. 对于低维空间的线性模型不可解问题，是否可以特征升维，转换为高维空间的线性可解问题。
3. 如果2成立，为什么还要研究非线性方法？成立，但只有理论意义！
4. 线性模型，使用固定基函数
    1. 全局的固定基函数不够灵活（例如多项式），维度太高后，无法解算，也很难获取足够训练数据；
    2. 局部的固定基函数，在没有见到训练数据之前，不知道中心点等信息，无从下手，思考RBF和Sigmoidal
5. 以神经网络为代表的的非线性方法，使用数据自适应关联的基函数
    1. 神经网络用数据的加权组合作为基函数，基函数数量固定，学习权值来获取最适合对当前数据的基函数。
    2. 支持向量机针对所有训练样本构建基于核函数的基函数，并学习得到一小部分作为最适合的基函数

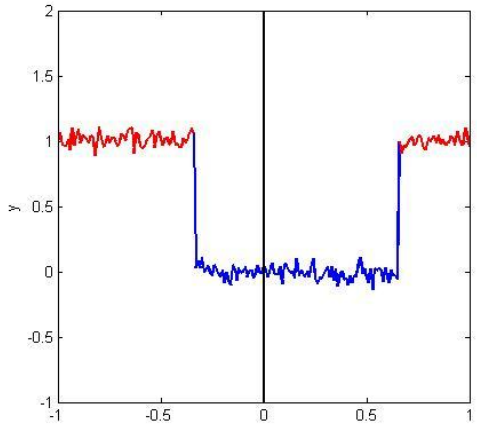
1 神经网络的出发点： 高维空间与线性可分

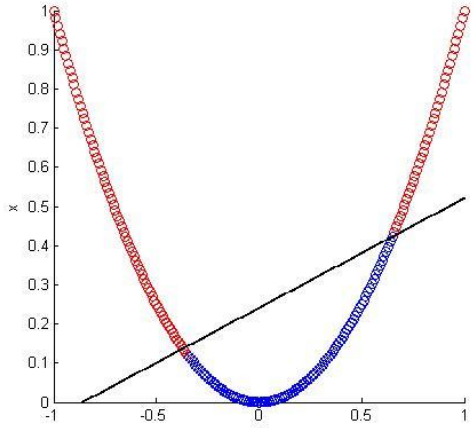




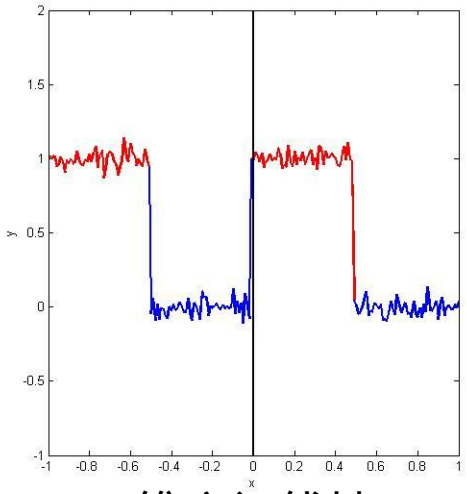
一维空间线性不可分，多项式函数升维后二维空间线性可分

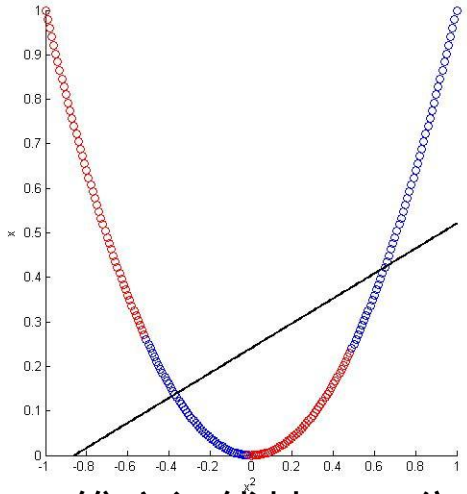
1 神经网络的出发点：高维空间与线性可分



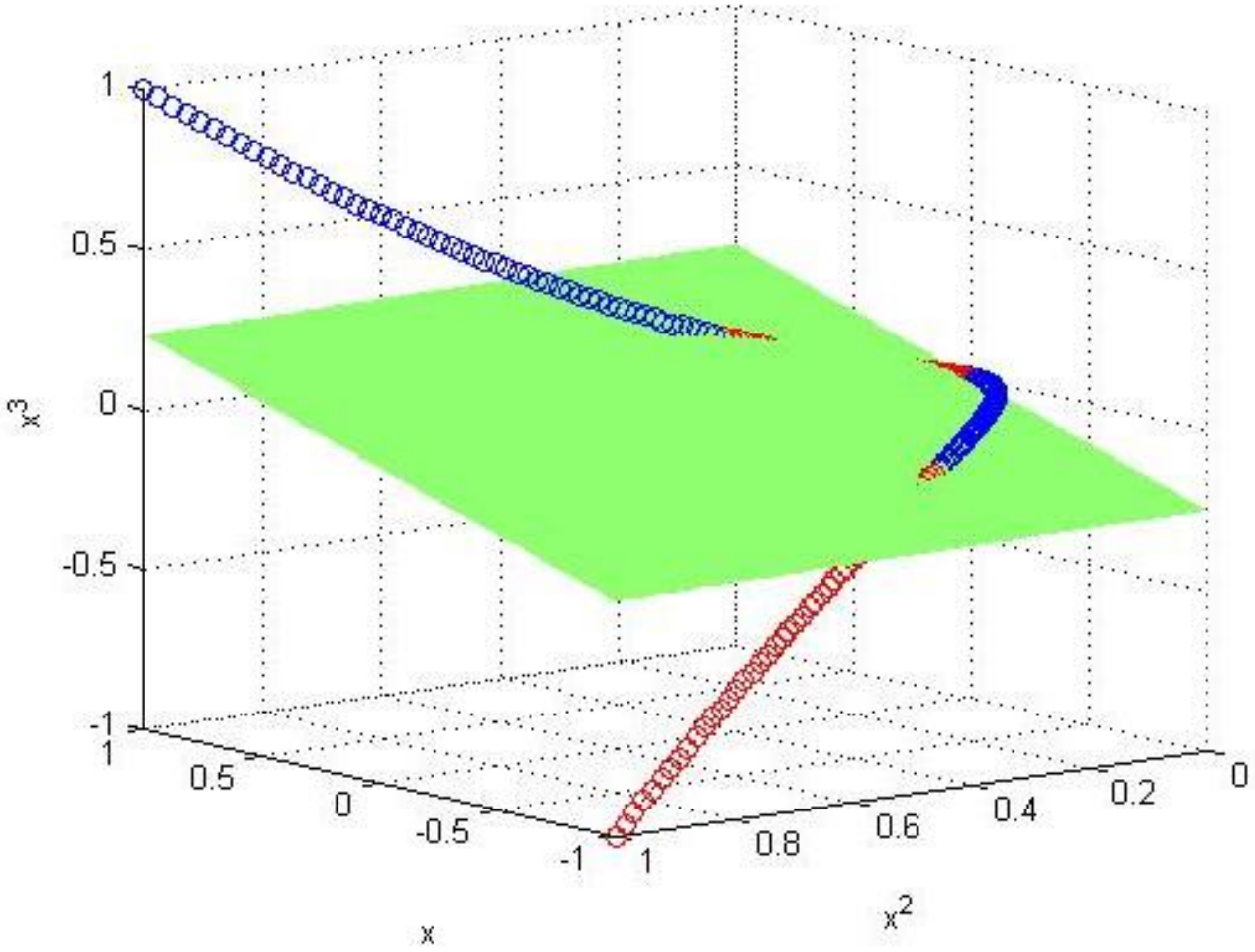


一维空间线性不可分，二维空间线性可分





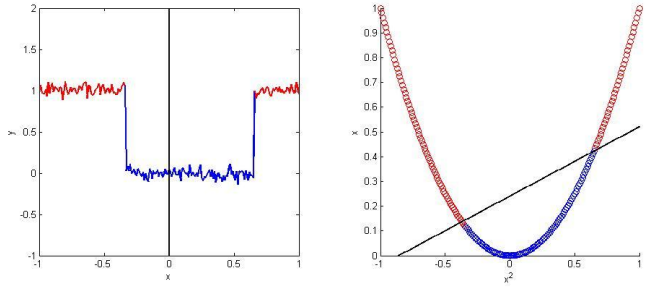
一维空间线性不可分，二维空间线性不可分

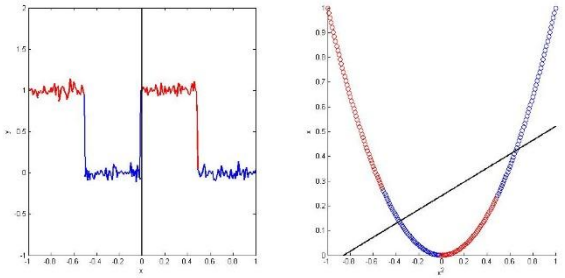


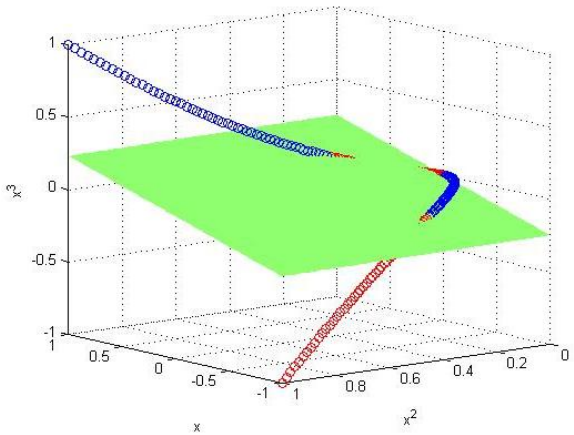
二维空间线性不可分，三维空间线性可分

1 神经网络的出发点：维数灾难

1. 数据特征维数过高的情况下，模型复杂度非常高，训练样本数量不变的情况下很容易过拟合。
   - ✓ 以多项式回归为例，数据维度D，多项式阶数M，独立参数量是$\boldsymbol{D^M}$
   - ✓ D=100，M=3，参数数量$\boldsymbol{10^6}$，需要的训练数据量非常巨大，数量有限的样本很容易过拟合
   - ✓ 更普遍的问题是“维度灾难”：数据稀疏（空心）、距离失效（样本间距离相似）等
2. 大部分实际问题要处理的数据集是高维空间中的一个非线性局部空间
   - ✓ 不需要很高的维度，但是要找到精简的有效维度（基函数）。
3. 因此要寻找更为精巧的方法

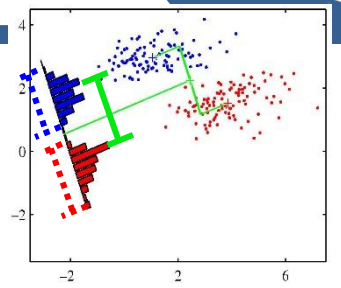


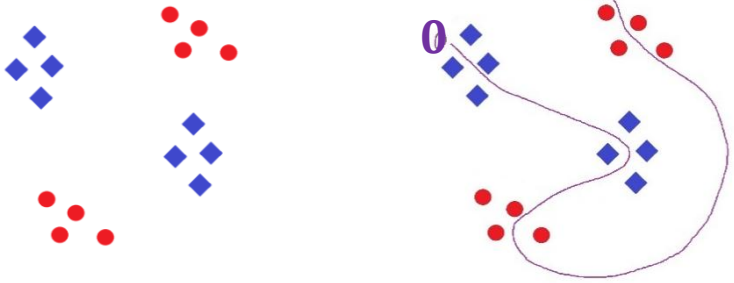




1 神经网络的出发点：高维空间中数据的分布

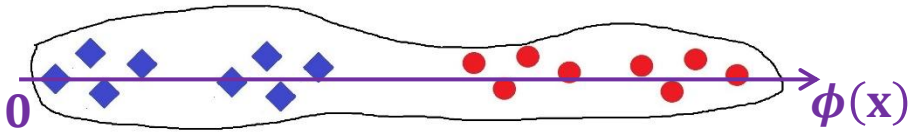
1. 数据的局部性
   - 很多应用中，数据不分布在整个空间，局限在一个低维局部空间
     - ✓ 低维流形 Manifold。
   - 用线性方法在高维空间建立模型计算困难、效率低、笨重。
2. 应直接学习（描述）嵌入在高维空间的低维流形
   - 找到一组最精简的基函数，能够重建这个流形中所有数据。
   - 流形中的数据在基函数上的系数就是其在流形中的坐标。
   - 用流形内的坐标计算欧式距离，实现回归和分类。
   - 不用原始空间内的欧式距离计算回归和分类。





2维空间不可分数据

嵌入在2维空间中的
1维可分流形



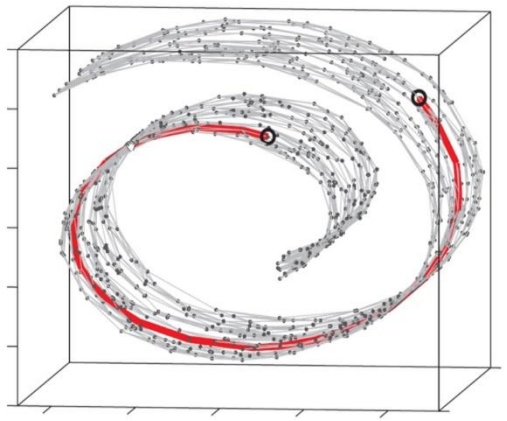
1维空间可分数据

流形内部的距离用欧氏距离度量，有点像重新排列了原来的数据，让t值接近的数据在$\phi(x)$空间上也接近或可分。

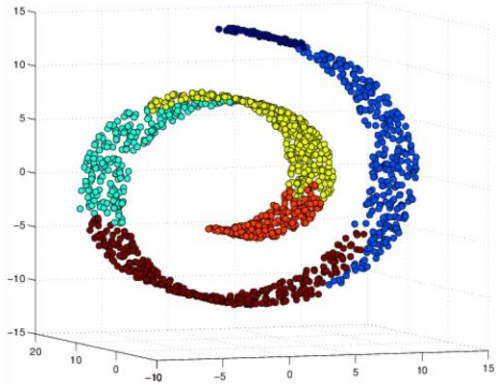
1 神经网络的出发点：高维空间中的低维流形

关键问题：如何定义距离，或如何定义子空间。即在哪些基函数张成的空间内计算欧式距离！

1. 二维平面的圆，如果数据都分布在圆上，则应该在一维空间定义直线距离
2. 三维平面，例如计算北京到上海的距离，三维欧式距离？球面距离？
   - ✓ 应该在二维空间（经纬为基函数）距离。经纬度就是低维流形上的坐标。
3. 数据的“瑞士卷”分布，典型的需要学习低维流形的问题。

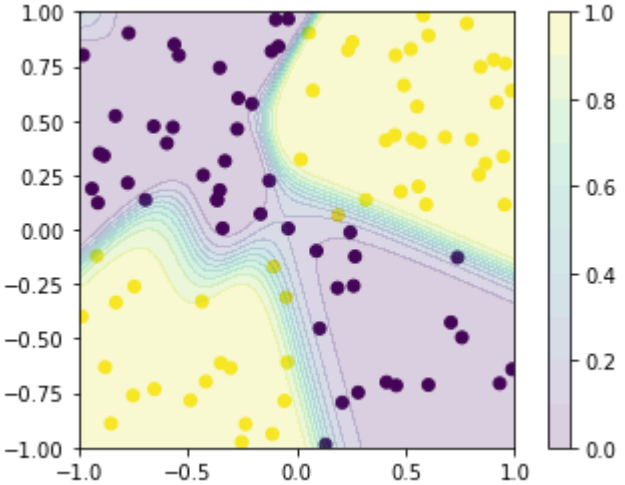


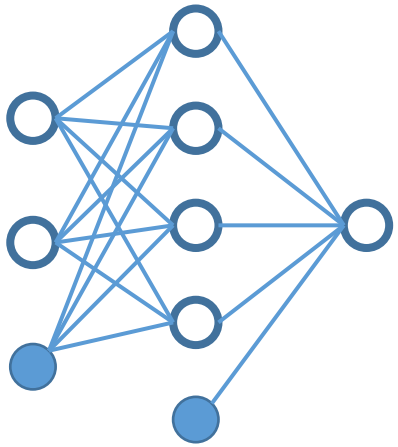




1 神经网络的出发点：数据自适应基函数

1. 数据的局部性
2. 应直接学习（描述）嵌入在高维空间的低维流形
3. 典型的非线性方法通过构建自适应关联数据的基函数，实现低维流形的描述
   - 神经网络：固定个数的基函数，每个基函数为数据的加权组合，权值可学习调整，基于参数的方法。
   - 支持向量机：可变个数的基函数，从所有训练数据的核函数中选择（学习），Memory -Based。





2 前馈神经网络函数

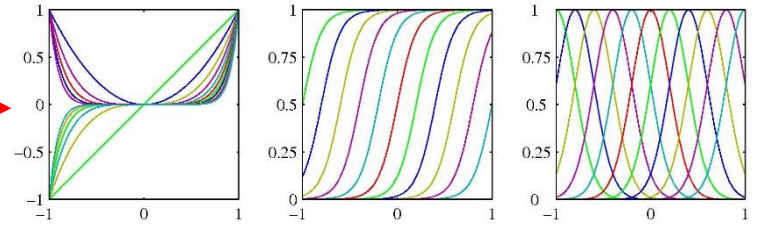
1. 前馈神经网络是使用最为广泛的神经网络
2. 典型结构为两层网络（隐含层+输出层），一般原始数据不算做一层。
3. 隐含层的输出本质上就是基函数的函数值
   - 隐含层的输出是原始数据加权和的激活结果，激活函数有Logistic、tanh、ReLU、Softplus、恒等（不激活）等
4. 输出层的输出是基函数值的线性加权组合的激活结果，与线性回归和Logistic回归完全一致
   - 如果是回归，激活函数就是恒等函数
   - 如果是两类分类，激活函数就是Sigmoid
   - 如果是多类分类，激活函数就是Softmax
5. 如果是隐含层多于1层，则基函数就是x的多重线性组合并激活的结果

2 前馈神经网络函数：基本结构

由线性模型扩展而来，形式可统一为： $y_k = f(\mathbf{w}_k{}^T \phi(\mathbf{x}))$

两类分类： $f(\mathbf{x}) = Sigmoid(\mathbf{x})$； 多类分类：$f(\mathbf{x}) = Softmax(\mathbf{x})$； 回归： $f(\mathbf{x}) = \mathbf{x}$

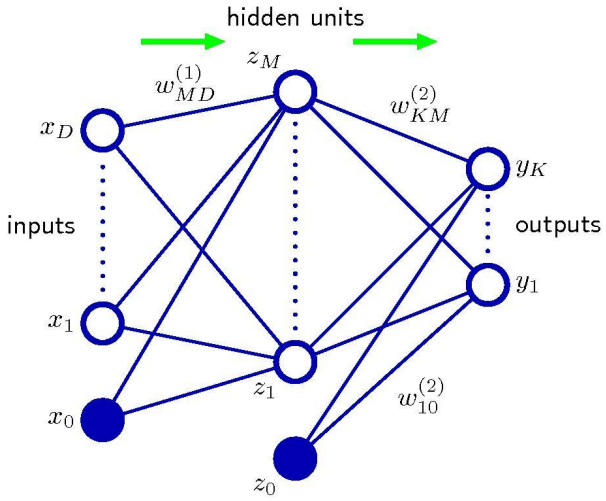
$$y_k = f(\mathbf{w}_k{}^T \phi_{Fixed}(\mathbf{x}))$$



1. 线性模型：基函数为固定函数，多项式、Sigmoidal、RBF

2. 神经网络：基函数为“x的加权线性组合的激活值”，加权系数w根据训练数据确定。

$$y_k = f(\mathbf{w}_k{}^T \phi_{NN}(\mathbf{x})) = f\left(\sum_{j=1}^{M} w_{kj}^{(2)} \underbrace{h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i\right)}_{\phi_{NN}(\mathbf{x})}\right)$$



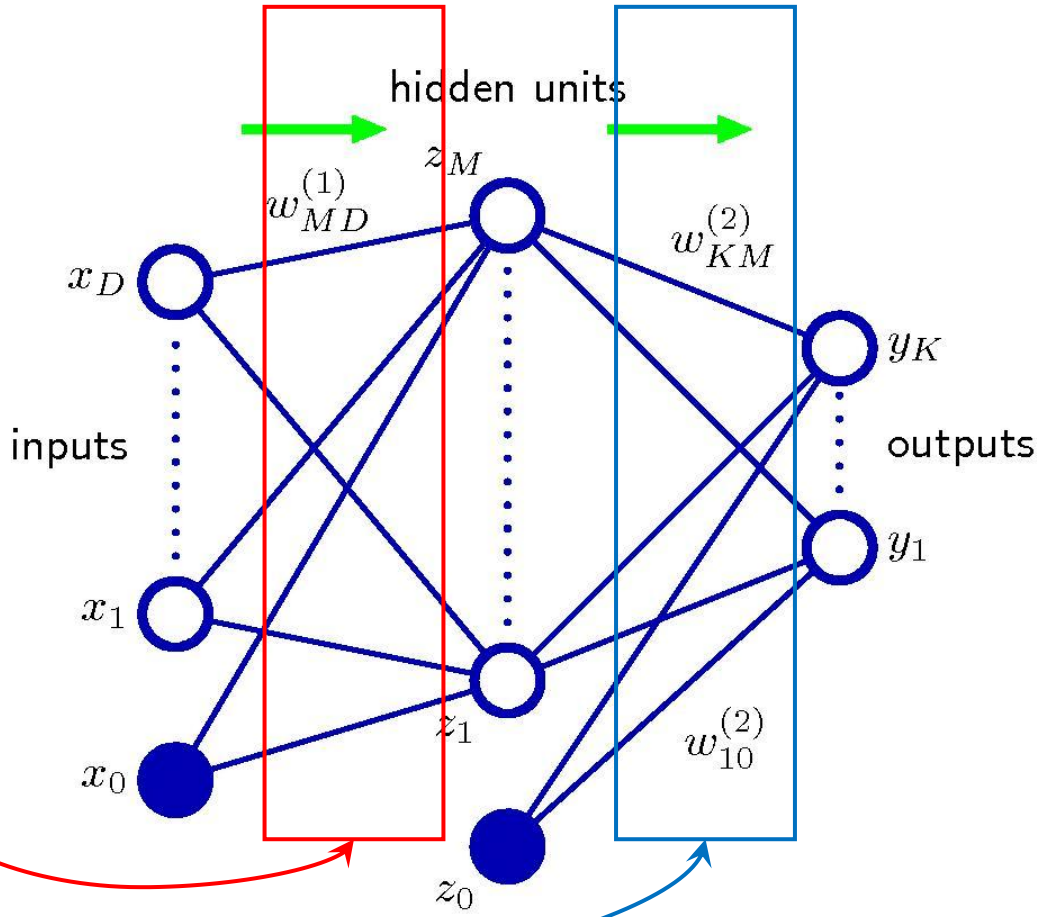
2 前馈神经网络函数：基本结构

1. 基函数为“x的加权线性组合的激活值”，加权系数w根据训练数据确定。

$$y_k = f\left(\mathbf{w}_k{}^T \phi(\mathbf{x})\right) = f\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i\right)\right)$$

2. 基函数加权组合并激活得到输出，与线性模型相同

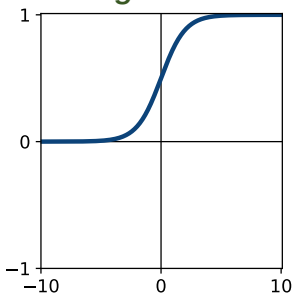
3. 神经网络的参数分两部分：
   2. 构成基函数的与x组合相乘的系数
   3. 构成输出的与基函数组合相乘的系数



2 前馈神经网络函数：隐含层激活函数

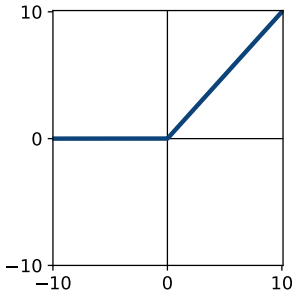
1. 梯度消失/衰减：
   - 线性分类模型普遍使用的Sigmoid激活函数，在x趋向正、负无穷大的时候，梯度接近0。
   - 神经网络求解需要使用梯度信息进行逐层传播，对于复杂神经网络，梯度消失有可能使相当一部分网络参数求解失败，无效神经元。
2. 常用的激活函数还有Sigmoid、ReLU等

Sigmoid



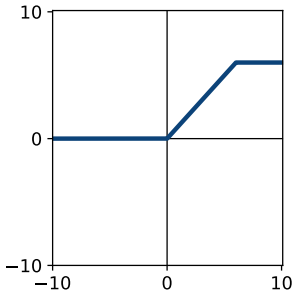
$y = 1/(1+e^{-x})$

ReLU



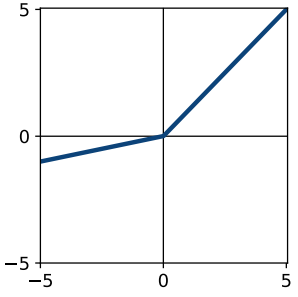
$y = \max(0, x)$

ReLU6



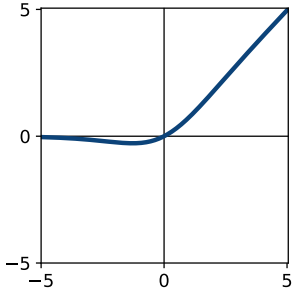
$y = \min(\max(0, x), 6)$

Leaky ReLU



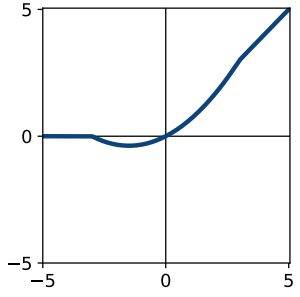
$y = \max(\alpha x, x)$

Swish



$y = x/(1+e^{-x})$

Hard Swish



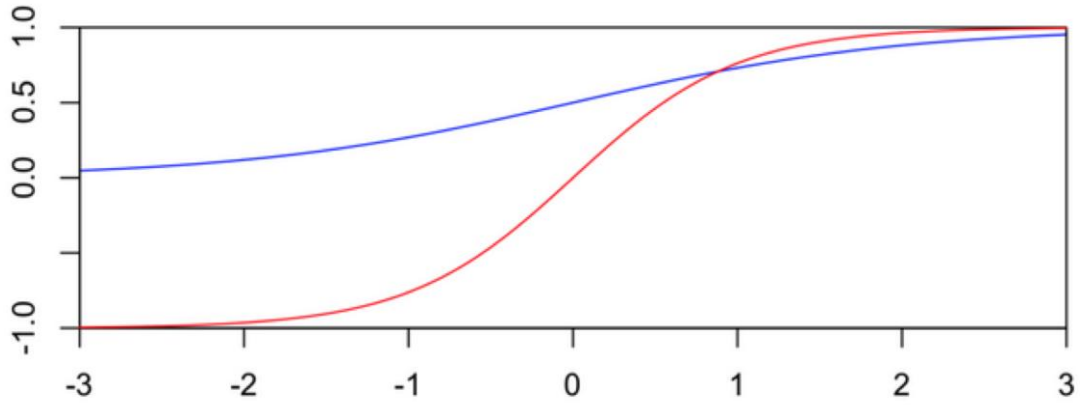
$$y = \begin{cases} 0 & x \le -3 \\ x & x \ge 3 \\ x \cdot (x+3)/6 & otherwise \end{cases}$$

2 前馈神经网络函数：激活函数

1. 激活函数$tanh$函数是拉伸的$sigmoid$函数

$$tanh(a) = 2\sigma(2a) - 1 = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}$$

2. [-1,1]输出对于神经网络更方便



sigmoid 和 tanh对比

2 前馈神经网络函数：激活函数

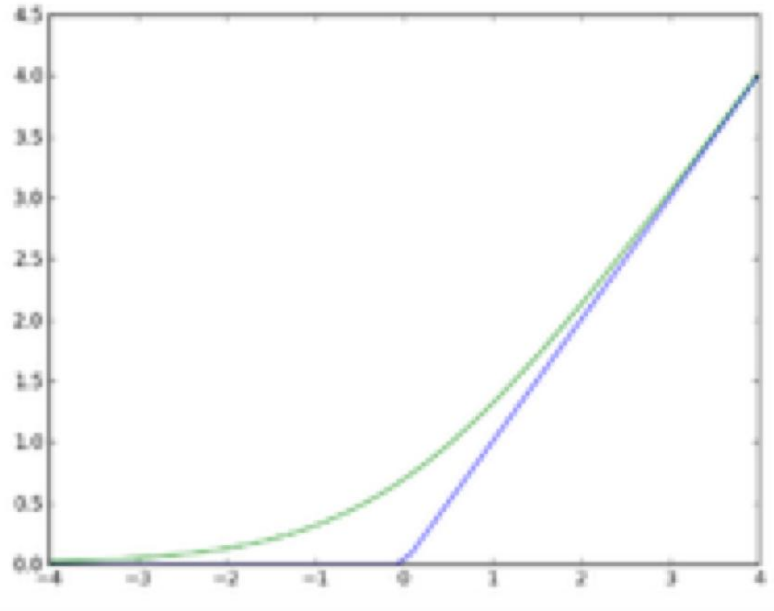
1. 激活函数ReLU函数是Rectified Linear Unit

$$ReLU(a) = max(0, a)$$

2. ReLU及其各种扩展在深度神经网络中普遍使用
3. 没有sigmoid和tanh的梯度消失问题，可支持更深的网络
4. ReLU的平滑逼近版本是softplus

$$tanh(a) = \ln(1 + e^{a}), tanh(a)' = \frac{1}{1 + e^{-a}}$$

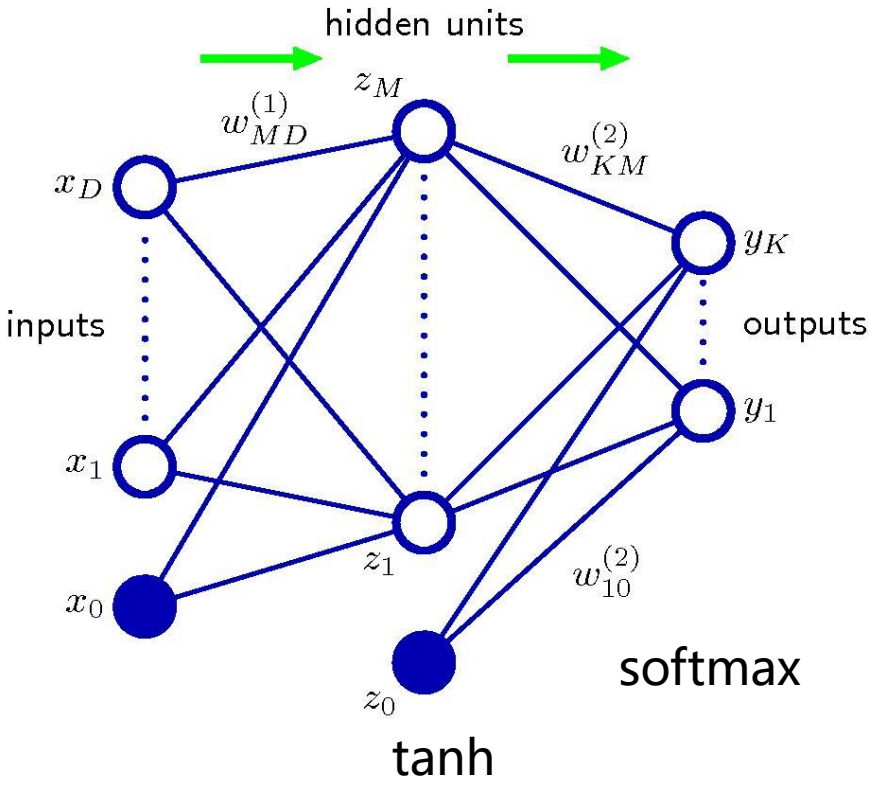
2. softplus的一阶导数是sigmoid函数

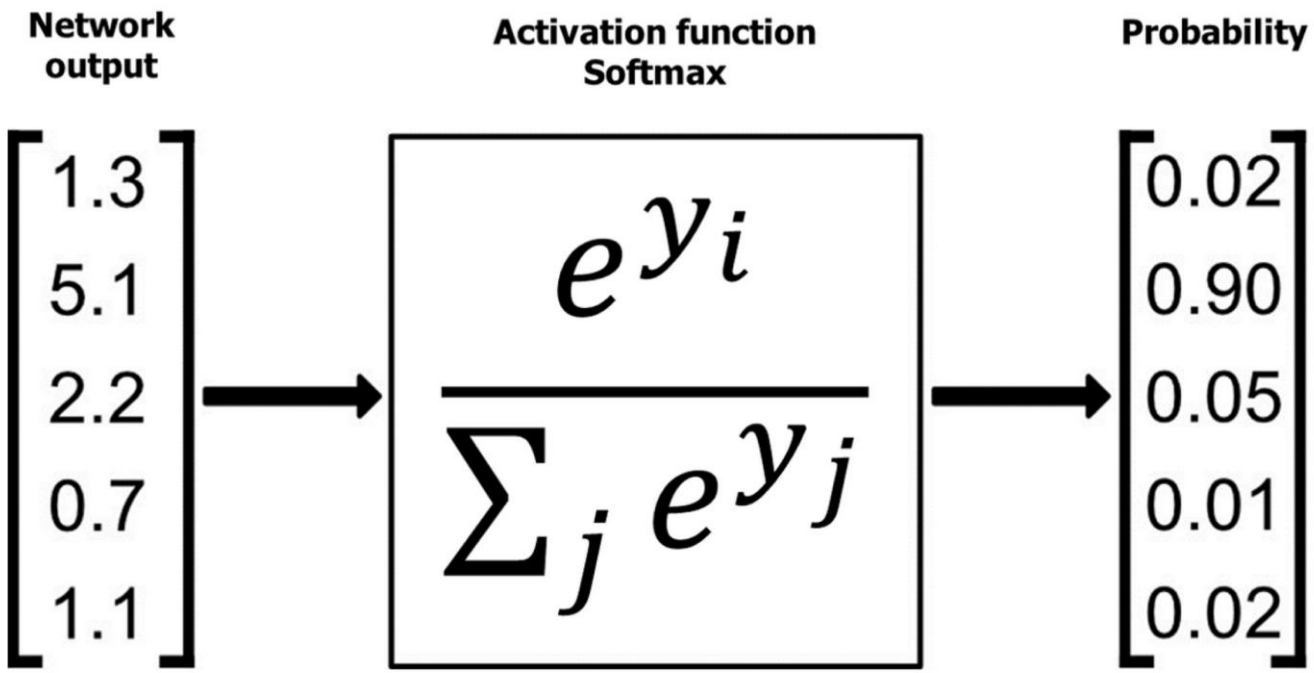


ReLU 和 softplus对比

2 前馈神经网络函数：输出层激活函数

- Sigmoid、Softmax、恒等
- 一个多类分类神经网络的例子，输出结果接近1-of-K





2 前馈神经网络函数：基本结构

- 回归：

$$y = \sum_{j=1}^{M} w^{(2)} h\left(\sum_{i=1}^{D} w^{(1)} x_i\right)$$

- 两类分类：

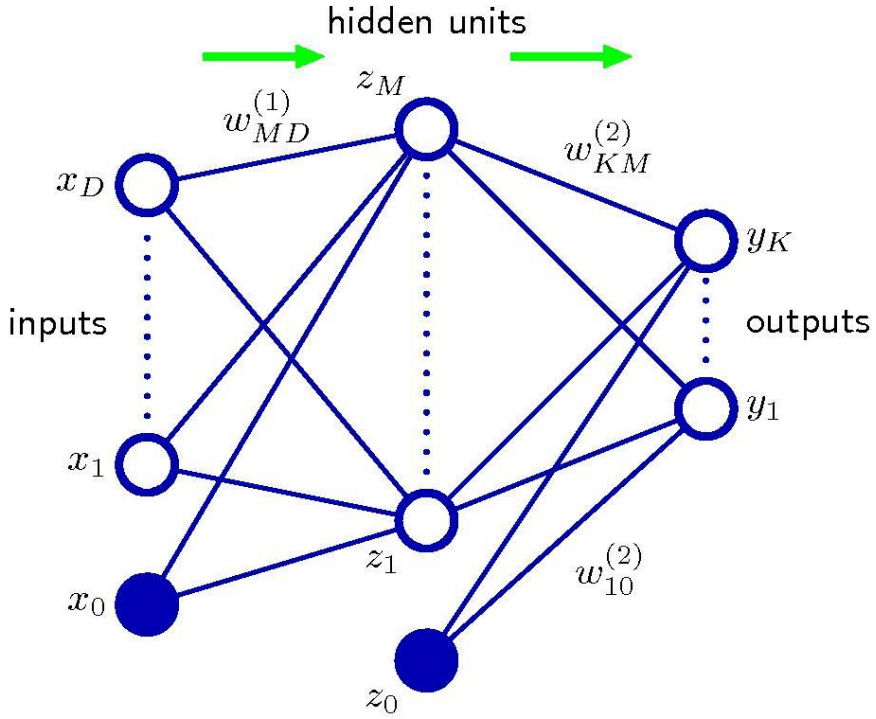
$$y = sigmoid\left(\sum_{j=1}^{M} w^{(2)} h\left(\sum_{i=1}^{D} w^{(1)} x_i\right)\right)$$

- 多类分类：

$$y_k = softmax\left(\sum_{j=1}^{M} w^{(2)} h\left(\sum_{i=1}^{D} w^{(1)} x_i\right)\right)$$

Hidden Units可以是多层

每多一层，基函数就多嵌套一次激活



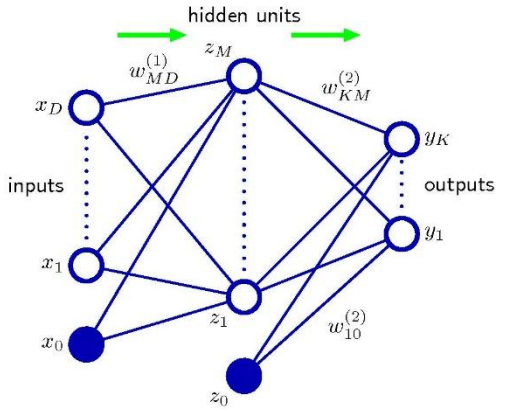
2 前馈神经网络函数：损失函数

- 用于回归的神经网络$y=\sum_{j=1}^{M} w^{(2)}\, h\left(\sum_{i=1}^{D} w^{(1)} x_i\right)$的损失函数

$$Loss(\mathbf{w})=\frac{1}{2}\sum_{n=1}^{N}(y_n-t_n)^2$$

- 用于两类分类的神经网络$y=sigmoid\left(\sum_{j=1}^{M} w^{(2)}\, h\left(\sum_{i=1}^{D} w^{(1)} x_i\right)\right)$的损失函数

$$\mathrm{Loss}(\mathbf{w})=-\sum_{n=1}^{N}\{t_n \ln y_n+(1-t_n)\ln(1-y_n)\}$$



- 用于多类分类的神经网络$y_k=softmax\left(\sum_{j=1}^{M} w^{(2)}\, h\left(\sum_{i=1}^{D} w^{(1)} x_i\right)\right)$的损失函数

$$\mathrm{Loss}(\mathbf{w})=-\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk}\ln\{y_{nk}\}$$

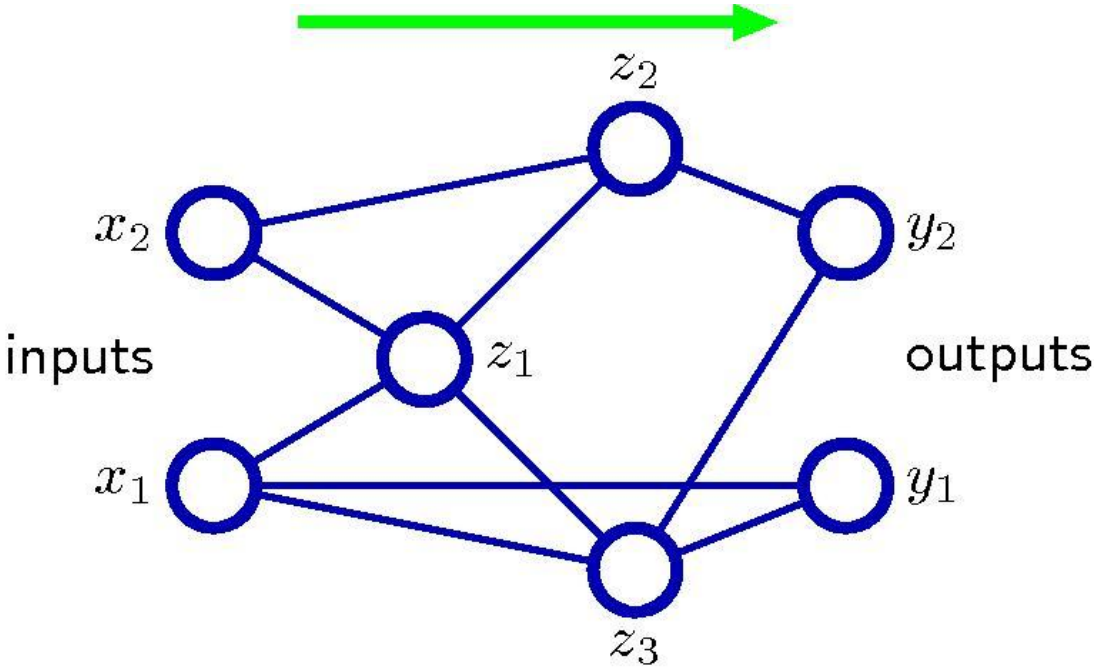
M个Hidden Units

$$\mathbf{h}(\mathbf{x})=\left(h_1(\mathbf{x}),h_2(\mathbf{x}),\cdots,h_M(\mathbf{x})\right)$$

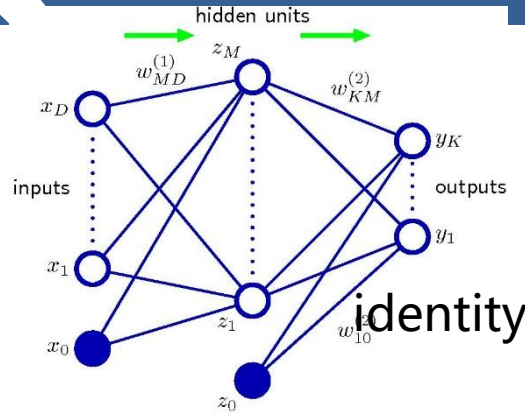
$$y_{nk}=\mathrm{softmax}(a_k)=\frac{\exp(a_k)}{\sum_j \exp(a_j)},\ a_k=\mathbf{w}_k^{(2)}\mathbf{h}(\mathbf{x})$$

2 前馈神经网络函数：稀疏节点

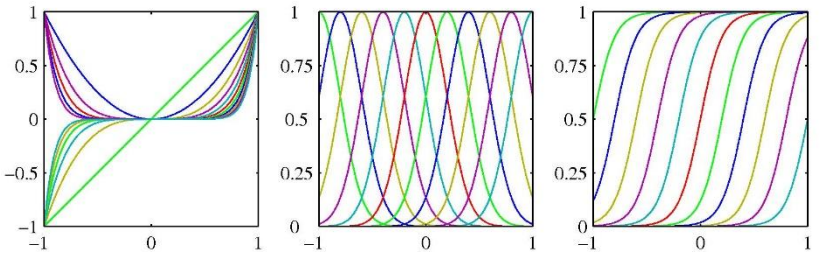
- 只要保持前馈结构，可以稀疏
- 不能有反向循环连接
- 输出只取决于输入
- 循环网络，例如Hopfield网络

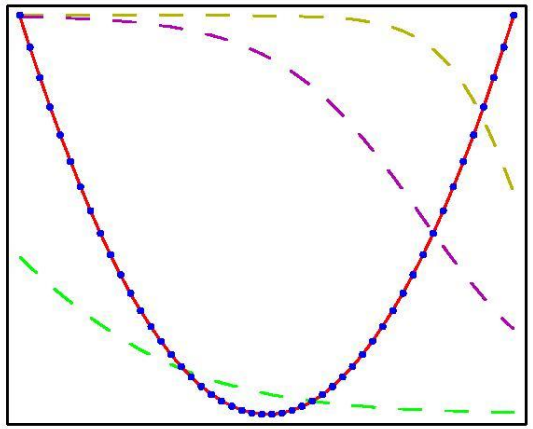


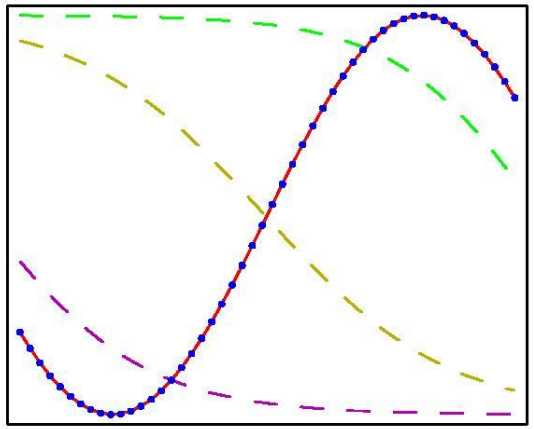
2 前馈神经网络函数：例子-回归

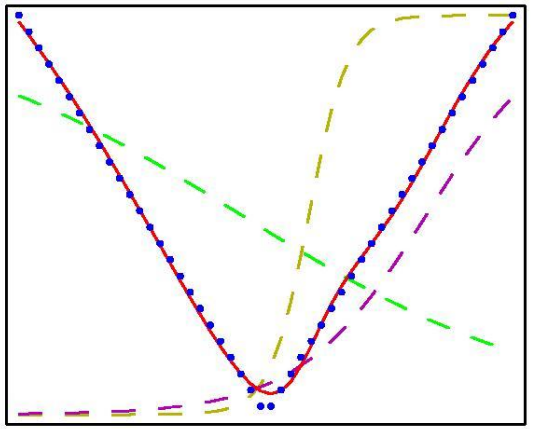


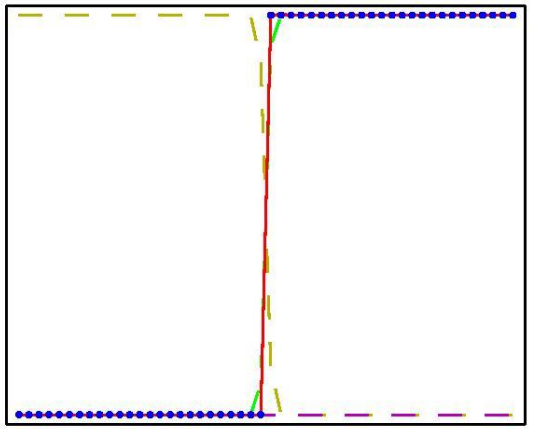
- 一个两层神经网络，隐含层有三个神经元，使用tanh激活函数
- 三个隐含层神经元协作逼近目标函数
- 虚线是三个隐含层的输出函数
- 对比线性回归中的固定基函数





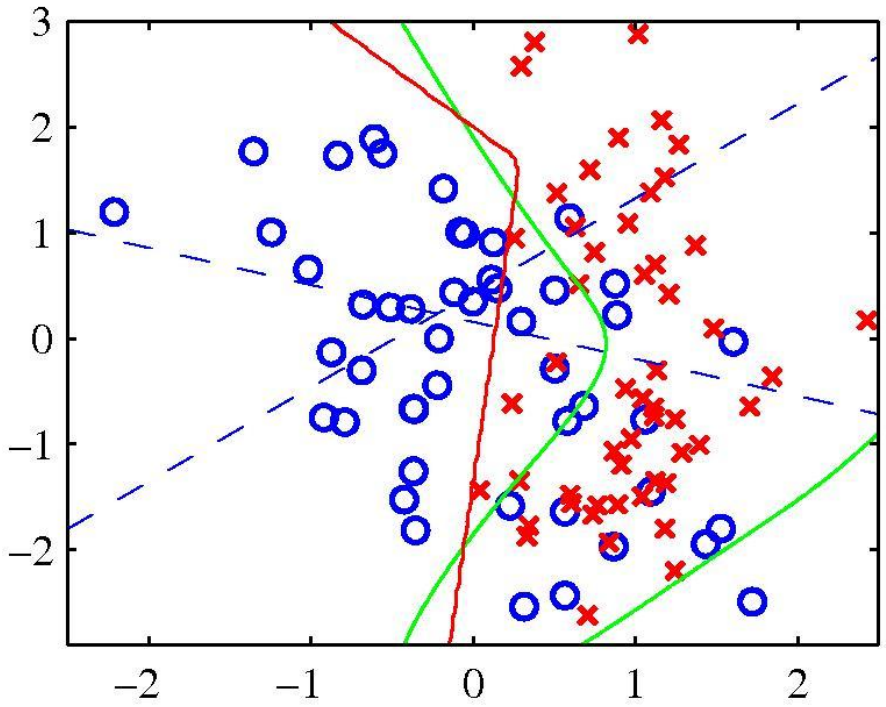




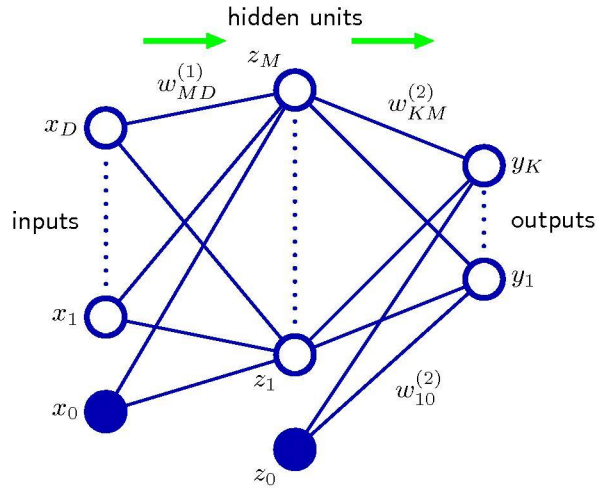


2 前馈神经网络函数：例子-两类分类

- 一个两层神经网络，隐含层有2个神经元，使用tanh激活函数
- 两个隐含层神经元协作逼近真实分类面（绿色）
- 虚线是两个隐含层的分类面
- 两个分类面激活组合出神经网络分类面



2 前馈神经网络函数：权值空间对称性/不可辨识性



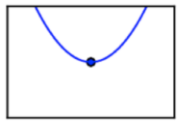
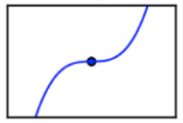
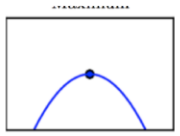
1. 多个w可以得到相同的输出，考虑隐含层使用tanh激活函数的情况
2. 对于隐含层某个神经元，把输入层到隐含层的权值改为异号。
3. 由于 tanh(-x)=-tanh(x)
4. 只需要把隐含层到输出层的权值也改为异号，神经网络输出就可保持不变。
5. 如果隐含层神经元数量为M
6. 每个w都是2^M个等价权（可得到相同输出）中的一个（每个hidden unit有2组w，共M个）。
7. 因为hidden unit的位置可交换，因此有M的全排列个组合，即M!
8. 每个w的等价权个数为$M!\,2^M$

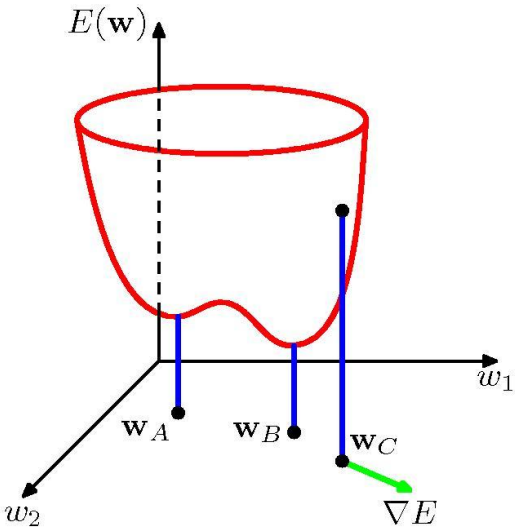
3 前馈神经网络的训练：线性模型损失函数

- 线性回归：平方差损失函数，凸，线性优化（是凸规划的一个特例）。
- Logistic回归：交叉熵损失函数，凸，非线性优化。极值为全局极小值。
- 凸：全程二阶导数>0，极值为全局极小值。
- 极值点(驻点)，一阶导数=0，二阶导数>0，极小值。

- 注意：如果Logistic回归使用预测值与观测值的平方差损失函数，由于激活函数的存在，损失函数将是非凸的，存在局部极小。

3 前馈神经网络的训练：线性模型损失函数

1. 回归，平方差损失函数
2. 分类，交叉熵损失函数
3. 基函数的非线性程度高，上述两者均为非线性、非凸（存在局部极小点）
4. 权值对称性（系统不可辨识性）也会导致多个相同的局部极小点$M!\,2^M$ 。
5. 依据Hessian矩阵正定性（$\mathbf{a}^T\mathbf{H}\mathbf{a} > 0$，a为任意向量） 判断驻点是否为极小点
   1. 极小点 
   2. 鞍点 
   3. 极大点 



$\mathbf{w}_A$ 局部极小

$\mathbf{w}_B$ 全局极小

$\mathbf{w}_C$ 处梯度指向函数值上升的方向

3 前馈神经网络的训练：梯度和海森矩阵

- 梯度向量（一阶导数）

$$\text{Loss}(\mathbf{w}), \mathbf{w} = (w_1, w_2)^T$$

- Hessian矩阵（二阶导数）

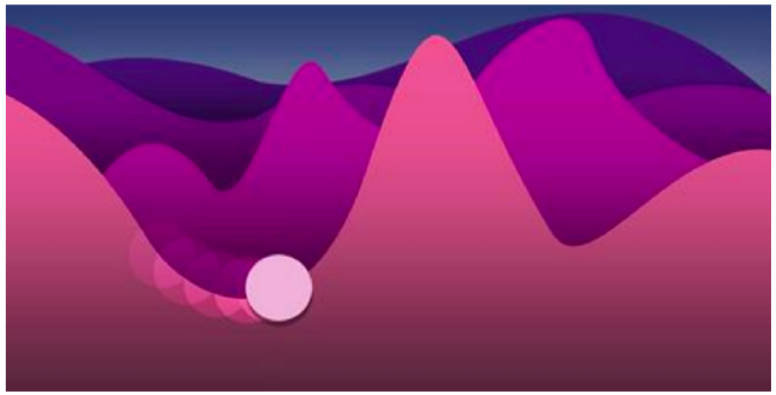
$$\nabla \text{Loss}(\mathbf{w}) = \frac{d\nabla \text{Loss}(\mathbf{w})}{d\mathbf{w}} = \begin{pmatrix} \frac{\partial \text{Loss}(\mathbf{w})}{\partial w_1} \\ \frac{\partial \text{Loss}(\mathbf{w})}{\partial w_2} \end{pmatrix}$$

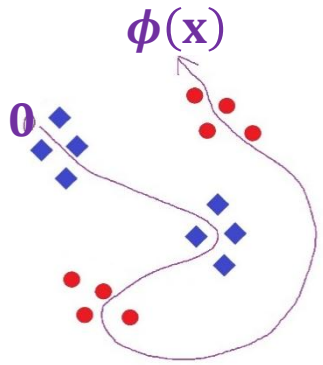
$$\mathbf{H} = \nabla\nabla \text{Loss}(\mathbf{w}) = \frac{d^2\nabla \text{Loss}(\mathbf{w})}{d\mathbf{w}^2} = \begin{pmatrix} \frac{\partial^2 \text{Loss}(\mathbf{w})}{\partial {w_1}^2} & \frac{\partial^2 \text{Loss}(\mathbf{w})}{\partial w_1 \partial w_2} \\ \frac{\partial^2 \text{Loss}(\mathbf{w})}{\partial w_2 \partial w_1} & \frac{\partial^2 \text{Loss}(\mathbf{w})}{\partial {w_2}^2} \end{pmatrix}$$

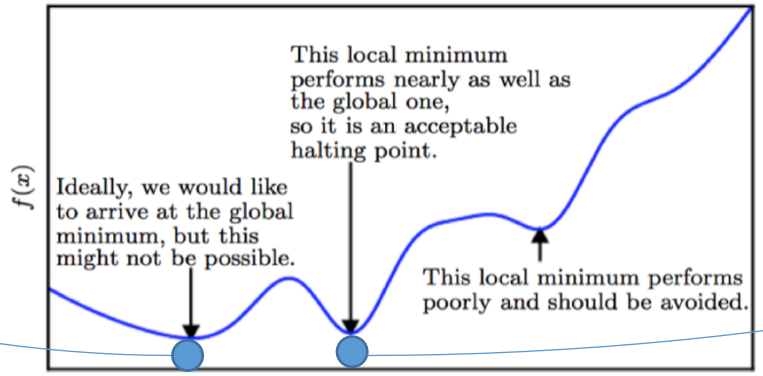
3 前馈神经网络的训练：全局极小与局部极小

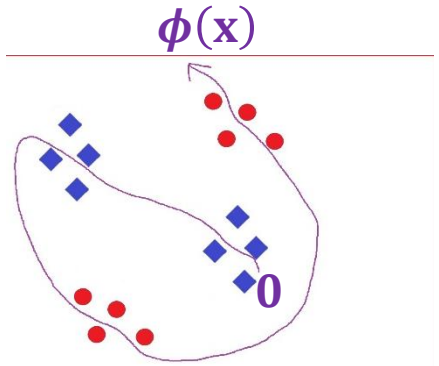
不一定非要找到全局极小，接近全局极小的局部极小值也可以

例如：学习到两个低维流形，但都能够完成欧式距离的分类和回归









3 前馈神经网络的训练：极小值的判断

极小值对应梯度为零，并且H正定，避开鞍点！

- **非零向量v**可以展开为基向量的线性组合 $\mathbf{v} = \sum_i c_i \mathbf{u}_i$，Hessian矩阵的二次型

$$\mathbf{v}^T \mathbf{H} \mathbf{v} = \left(\sum_i c_i \mathbf{u}_i \right)^T \mathbf{H} \sum_i c_i \mathbf{u}_i = \sum_i \lambda_i {c_i}^2$$

- 如果需要所有c_i都能使上式$\mathbf{v}^T \mathbf{H} \mathbf{v} > 0$成立，H的特征值必须全部为正。
- 即，当且仅当H的特征值都为正时，H正定！

3 前馈神经网络的训练：离线与在线方法

整体（整批）优化方法，每次更新模型都使用所有训练数据

1. 梯度下降法
2. 共轭梯度法
3. 拟牛顿法
4. 牛顿法

在线优化方法，每次只是用小部分（Mini-Batch）或1个（Mini-Batch=1）训练数据

1. 随机梯度下降法（SGD）
   a) 不受数据冗余的影响，因为不需要遍历所有数据，而是每个（或Mini-Batch）数据都更新模型
   b) 可逃离局部极小，因为全体数据集的损失函数的局部极小通常不是当前Mini-Batch数据的局部极小

3 前馈神经网络的训练：SGD不受冗余数据影响

考虑一种数据冗余的极端情况：把所有训练数据都复制一份，添加到训练集中。

- 以所有数据的整体损失作为梯度更新依据的算法，梯度的计算量加倍。

$$\mathrm{Loss}(\mathbf{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K} y_{nk}\ln\{\mathrm{softmax}(a_{nk})\},\ \nabla\mathrm{Loss}(\mathbf{w}) = \sum_{n=1}^{N}(\sigma_n - y_n)\boldsymbol{\phi}(\mathbf{x}_n)$$

- 但是随机梯度下降法不会受此影响（考虑mini-batch=1的情况）：

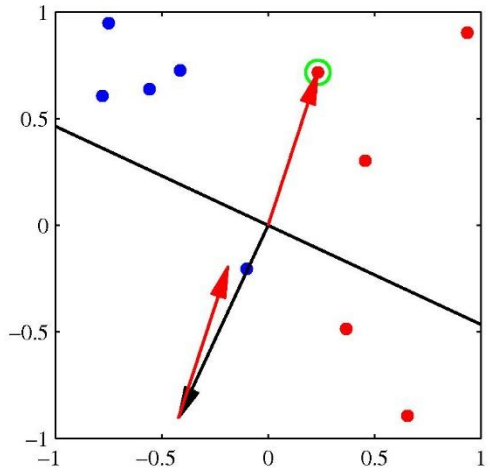
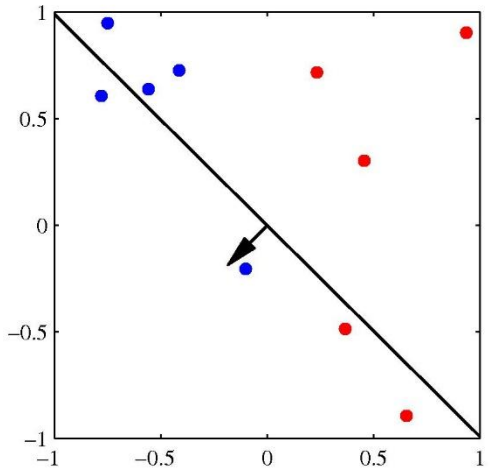
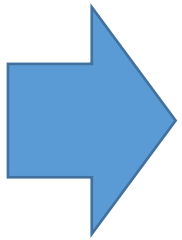
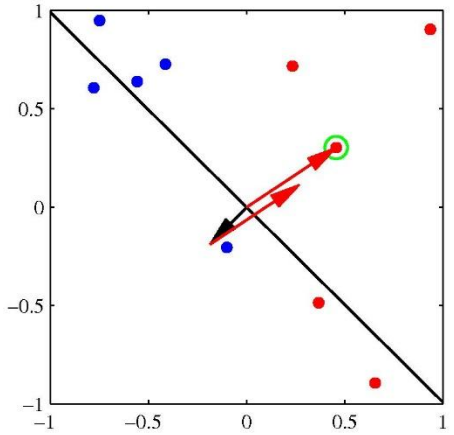
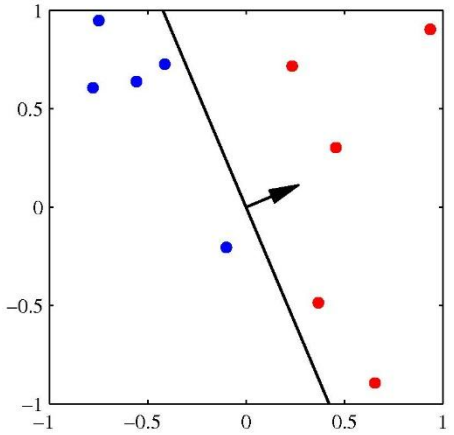
  a) 随机选取，可一定程度避免重复使用同样的数据；

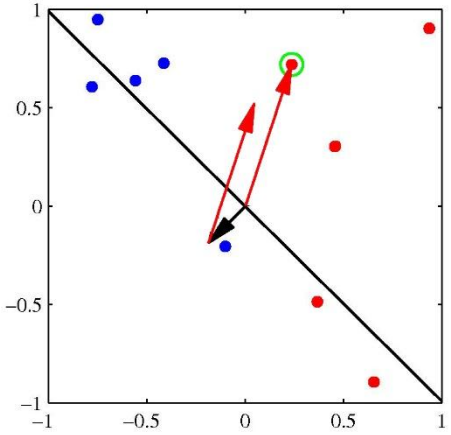
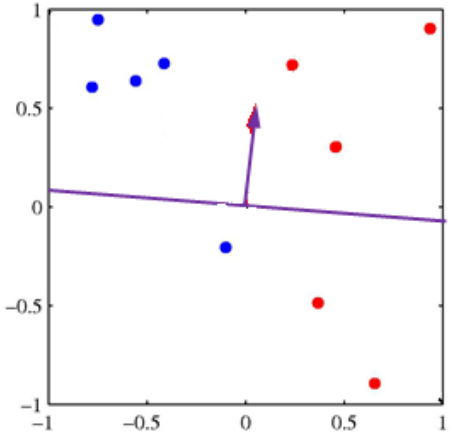
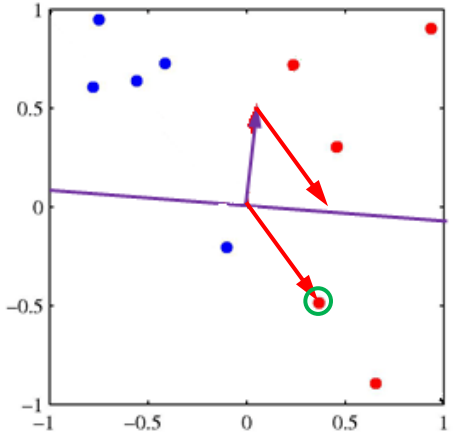
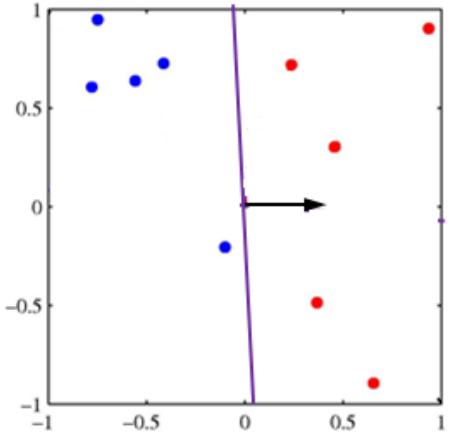
  b) 即使连续两次计算同一数据$(\mathbf{x}, y)$，每次计算也都会对神经网络权值（分类曲面）有贡献

  c) 使用$(\mathbf{x}, y)$更新模型后，如果$\mathbf{x}$已经正确分类，则第二次更新时，梯度为0，权值不更新

  d) 使用$(\mathbf{x}, y)$更新模型后，如果$\mathbf{x}$仍然错误分类，则第二次会继续更新，使分类面进一步变好。

3 前馈神经网络的训练： SGD不受冗余数据影响

连续两次使用

同一个数据更新权值

3 前馈神经网络的训练：SGD可逃离局部极小

- 整体损失函数

$$\mathrm{Loss}(\mathbf{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk}\ln\{\mathrm{softmax}(a_{nk})\}$$

在局部极小$\mathbf{w}_{Minima}$处，$\nabla\mathrm{Loss}(\mathbf{w}) = \sum_{n=1}^{N}(\sigma_n - t_n)\boldsymbol{\phi}(\mathbf{x}_n) = 0$

- 局部损失函数（单个点或mini-batch）

$$\mathrm{Loss}(\mathbf{w})_n = -\sum_{k=1}^{K} t_{nk}\ln\{\mathrm{softmax}(a_{nk})\}$$

在$\mathbf{w}_{Minima}$处，$\nabla\mathrm{Loss}(\mathbf{w})_n = (\sigma_n - t_n)\boldsymbol{\phi}(\mathbf{x}_n)$不一定为0

4 误差后向传播

- 用于多类分类的神经网络

$$y_k = softmax\left(\sum_{j=1}^{M} w^{(2)}\, h\left(\sum_{i=1}^{D} w^{(1)} x_i\right)\right)$$

- 损失函数

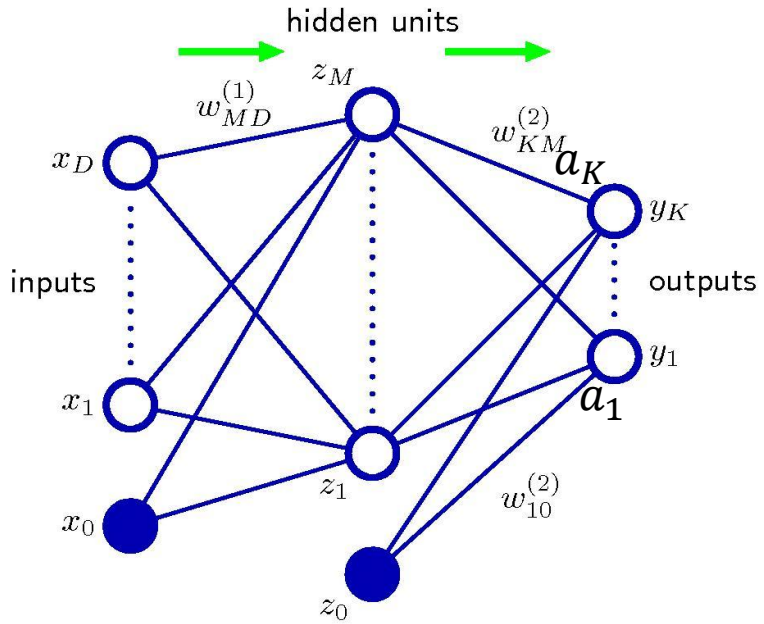
$$\mathrm{Loss}(\mathbf{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk}\ln\{y_{nk}\}$$

- 单个数据的损失，可省略下标n

$$\mathrm{Loss}(\mathbf{w}) = -\sum_{k=1}^{K} t_{nk}\ln\{y_{nk}\} = -\sum_{k=1}^{K} t_k\ln\{y_k\}$$

4 误差后向传播

- 用于多类分类的神经网络



$$y_k = softmax\left(\sum_{j=1}^{M} w_{kj}^{(2)}\, h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i\right)\right)$$

- 单个数据的损失，可省略下标n

$$\mathrm{Loss}_n(\mathbf{w}) = -\sum_{k=1}^{K} t_{nk}\ln\{y_{nk}\} = -\sum_{k=1}^{K} t_k \ln\{y_k\}$$

$$a_k = \sum_{j=1}^{M} w_{kj}^{(2)} z_j$$

$$\frac{\partial \mathrm{Loss}_n(\mathbf{w})}{\partial w_{kj}^{(2)}} = \frac{\partial \mathrm{Loss}_n(\mathbf{w})}{\partial a_k}\frac{\partial a_k}{\partial w_{kj}^{(2)}} = (y_k - t_k) z_j$$

此结果也适用于回归和两类分类，自行证明

4 误差后向传播

$$p(C_k|\mathbf{x}) = \text{softmax}(a) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}, a_k = \mathbf{w}_k{}^{\mathrm{T}}\phi(\mathbf{x})$$

$$\mathbf{w}_k = (w_0, w_1, w_2, \cdots, w_M)^{\mathrm{T}}$$

$$y_k = softmax(a_k)$$

$$\text{Loss}_n(\mathbf{w}) = -\sum_{k=1}^{K} t_k \ln\{y_k\} = -\sum_{k=1}^{K} t_k \ln\{softmax(a_k)\}$$

$$\frac{\partial \text{softmax}(a_k)}{\partial a_j} = \text{softmax}(a_k)\left(I_{kj} - \text{softmax}(a_j)\right) \quad I_{kj} = \begin{cases} 1, k = j \\ 0, k \neq j \end{cases}$$

$$\frac{\partial \text{Loss}_n(\mathbf{w})}{\partial w_{kj}^{(2)}} = \frac{\partial \text{Loss}_n(\mathbf{w})}{\partial a_k} \frac{\partial a_k}{\partial w_{kj}^{(2)}}$$

$$\sum_{k=1}^{K} t_k = 1 \rightarrow t_i = 1 - \sum_{k \neq i}^{K} t_k$$

$$\frac{\partial \text{Loss}_n(\mathbf{w})}{\partial a_i} = -\sum_{k=1}^{K} t_k \frac{1}{\text{softmax}(a_k)} \{\text{softmax}(a_k)(I_{ki} - \text{softmax}(a_i))\}$$

$$= -\sum_{k=1}^{K} t_k (I_{ki} - \text{softmax}(a_i)) = -\left\{ t_i (1 - \text{softmax}(a_i)) + \sum_{k \neq i}^{K} -t_k \text{softmax}(a_i) \right\}$$

$$= \text{softmax}(a_i)\left(\sum_{k \neq i}^{K} t_k\right) - t_i(1 - \text{softmax}(a_i)) = \text{softmax}(a_i)(1 - t_i) - t_i + t_i \text{softmax}(a_i) = \text{softmax}(a_i) - t_i = y_i - t_i$$

4 误差后向传播：输出层权值的梯度

$$\frac{\partial L}{\partial w_{kj}^{(2)}}$$

$$y_k = softmax\left(\sum_{j=1}^{M} w_{kj}^{(2)}\, h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i\right)\right)$$

$$\mathrm{Loss}_n(\mathbf{w}) = -\sum_{k=1}^{K} t_k \ln\{y_k\} \qquad y_k = softmax(a_k)$$

$$\frac{\partial \mathrm{Loss}_n(\mathbf{w})}{\partial w_{kj}^{(2)}} = \frac{\partial \mathrm{Loss}_n(\mathbf{w})}{\partial a_k}\frac{\partial a_k}{\partial w_{kj}^{(2)}} = (y_k - t_k) z_j \qquad a_k = \sum_{j=1}^{M} w_{kj}^{(2)} z_j$$

$$z_j = h(a_j)$$

输出层的误差：

$$\delta_k = \frac{\partial \mathrm{Loss}_n(\mathbf{w})}{\partial a_k} = y_k - t_k$$

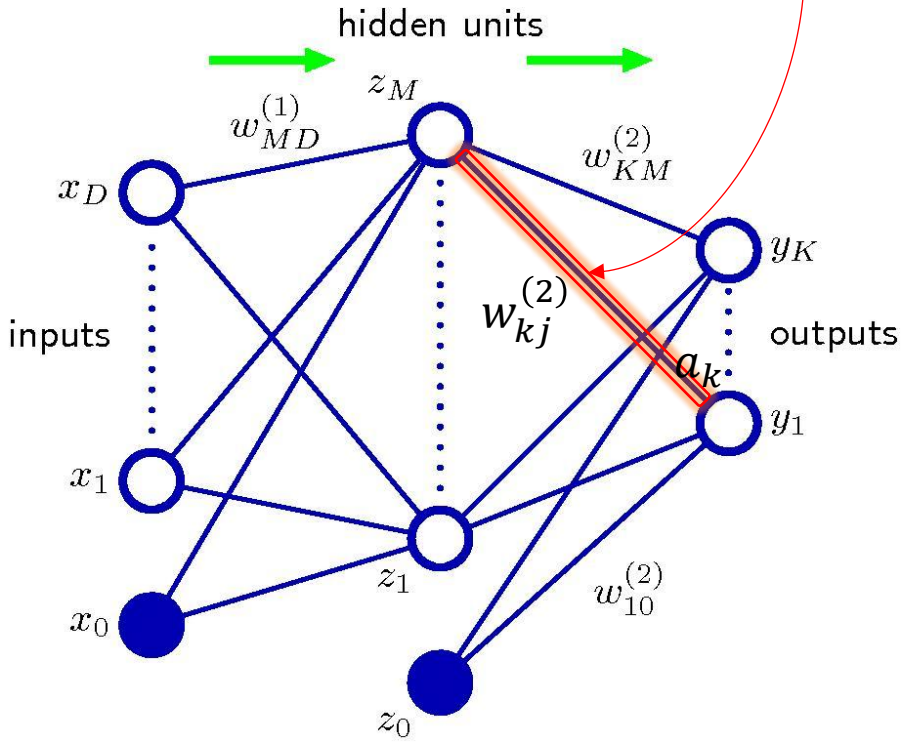
输出层的权值的导数：

$$\frac{\partial \mathrm{Loss}_n(\mathbf{w})}{\partial w_{kj}^{(2)}} = \delta_k z_j$$

$$\delta_k = y_k - t_k$$

Forward

$$y_k = softmax(a_k)$$



4 误差后向传播：隐含层权值的梯度

$$f_1(\mathrm{x}), \qquad f_2(\mathrm{x}), \qquad f_3(\mathrm{x})$$

$$L(f_1, f_2, f_3) = \lambda_1 \ln\big(f_1(\mathrm{x})\big) + \lambda_2 \ln\big(f_2(\mathrm{x})\big) + \lambda_3 \ln\big(f_3(\mathrm{x})\big)$$

全微分公式：

$$\frac{dL(\mathrm{x})}{dx} = \frac{\partial L}{\partial f_1}\frac{\partial f_1}{\partial x} + \frac{\partial L}{\partial f_2}\frac{\partial f_2}{\partial x} + \frac{\partial L}{\partial f_3}\frac{\partial f_3}{\partial x}$$

4 误差后向传播： 隐含层权值的梯度

$$\frac{\partial L}{\partial w_{ji}^{(1)}}$$

把Loss展开为a_k的表达

$$\text{Loss}_n(\mathbf{w}) = -\sum_{k=1}^{K} t_k \ln\{y_k\} = -\sum_{k=1}^{K} t_k \ln\{softmax(a_k)\}, a_k = \sum_{j=1}^{M} w_{kj}^{(2)} h(a_j)$$

$$\text{Loss}_n(\mathbf{w}) = L(a_1, a_2, \ldots, a_K); a_k = a_k(a_j) \qquad a_j = \sum_{i=1}^{D} w_{ji}^{(1)} x_i$$

全微分公式：

$$\frac{\partial L}{\partial a_j} = \sum_{k=1}^{K} \frac{\partial L}{\partial a_k}\frac{\partial a_k}{\partial a_j} = \sum_{k=1}^{K} \delta_k \left\{w_{kj}^{(2)} h'(a_j)\right\} = h'(a_j)\sum_{k=1}^{K} w_{kj}^{(2)}\delta_k = \delta_j$$

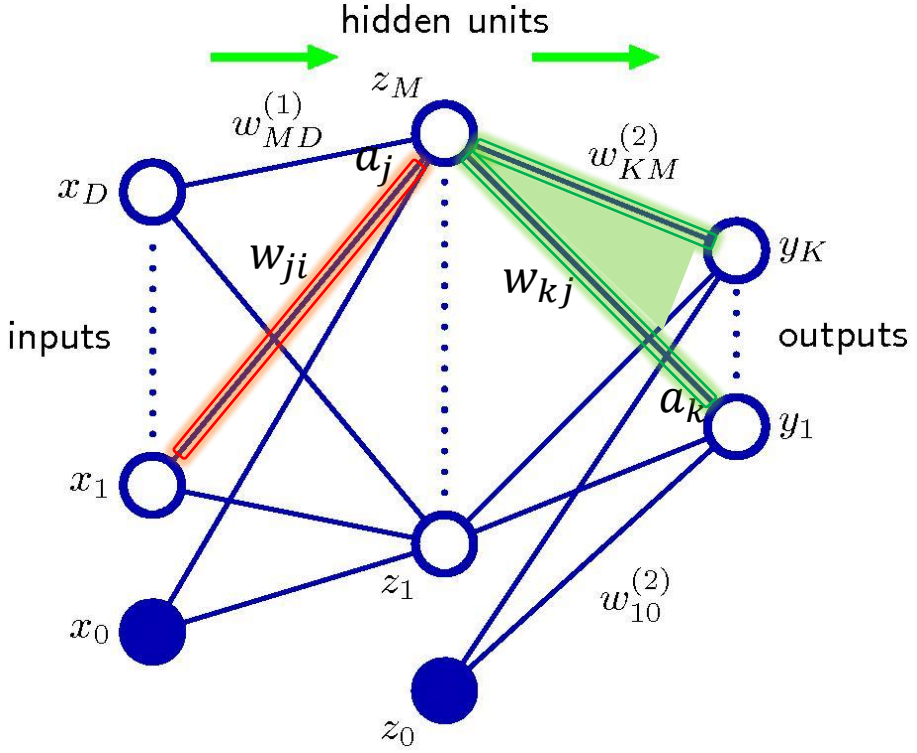
$$\frac{\partial L}{\partial a_k} = \delta_k = y_k - t_k, \qquad a_j = \sum_{i=1}^{D} w_{ji}^{(1)} x_i$$

$$\frac{\partial L}{\partial w_{ji}^{(1)}} = \frac{\partial L}{\partial a_j}\frac{\partial a_j}{\partial w_{ji}^{(1)}} = \left\{h'(a_j)\sum_{k=1}^{K} w_{kj}^{(2)}\delta_k\right\} x_i = \delta_j x_i$$

$$\delta_j = h'(a_j)\sum_{k=1}^{K} w_{kj}^{(2)}\delta_k$$

Forward

$$y_k = softmax\left(\sum_{j=1}^{M} w_{kj}^{(2)}\, h(a_j)\right)$$



4 误差后向传播：多个隐含层权值的梯度递归

$$\frac{\partial L}{\partial w_{ip}^{(0)}}$$

$$y_k = softmax(a_k)$$

$$a_k = \sum_{j=1}^{M} w_{kj}^{(2)} h(a_j)$$

$$a_j = \sum_{i=1}^{D} w_{ji}^{(1)} h(a_i)$$

$$a_i = \sum_{p=1}^{P} w_{ip}^{(0)} x_p$$

Forward

$$y_k = softmax\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} h\left(\sum_{p=1}^{P} w_{ip}^{(1)} x_p\right)\right)\right)$$

把Loss展开为a_j的表达式

$$\text{Loss}_n(\mathbf{w}) = -\sum_{k=1}^{K} t_k \ln\{y_k\} = -\sum_{k=1}^{K} t_k \ln\left\{softmax\left(\sum_{j=1}^{M} w_{kj}^{(2)} h(a_j)\right)\right\}$$

$$\text{Loss}_n(\mathbf{w}) = L\left(a_1^{(1)}, a_2^{(1)}, \ldots, a_M^{(1)}\right); \quad a_j = a_j(a_i)$$

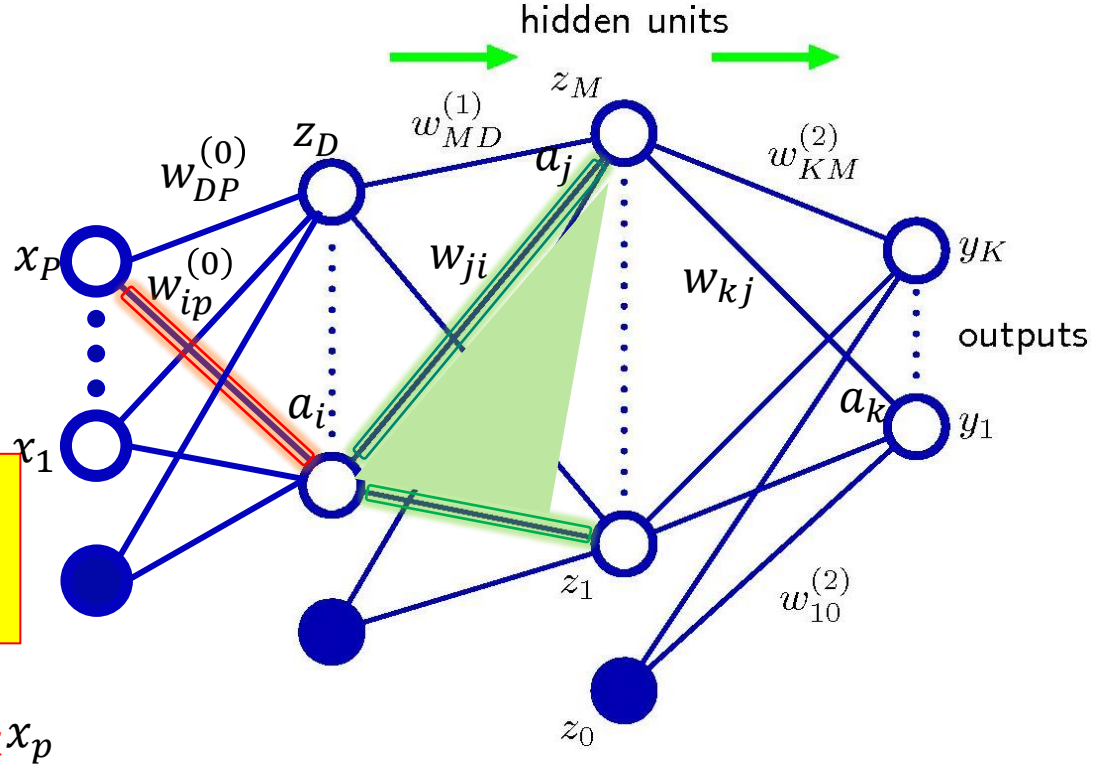
a_j

全微分公式：

$$\frac{\partial L}{\partial a_i} = \sum_{j=1}^{M} \frac{\partial L}{\partial a_j}\frac{\partial a_j}{\partial a_i} = \sum_{j=1}^{M} \delta_j \left\{w_{ji}^{(1)} h'(a_i)\right\} = \delta_i = h'(a_i)\sum_{j=1}^{M} w_{ji}^{(1)}\delta_j$$

$$\delta_j = \frac{\partial L}{\partial a_j} = h'(a_j)\sum_{k=1}^{K} w_{kj}^{(2)}\delta_k$$

$$\frac{\partial L}{\partial w_{ip}^{(0)}} = \frac{\partial L}{\partial a_i}\frac{\partial a_i}{\partial w_{ip}^{(0)}} = \delta_i x_p$$



4 误差后向传播：多个隐含层权值的梯度递归

输出层k

$$\delta_k = \frac{\partial L}{\partial a_k} = y_k - t_k$$

隐含层j=k-1

$$\delta_j = \frac{\partial L}{\partial a_j} = h'(a_j) \sum_{k=1}^{K} w_{kj}^{(2)} \delta_k$$

隐含层i=j-1

$$\delta_i = \frac{\partial L}{\partial a_i} = h'(a_i) \sum_{j=1}^{M} w_{ji}^{(1)} \delta_j$$

……

4 误差后向传播： 多个隐含层权值的梯度递归

数据前向计算 Forward

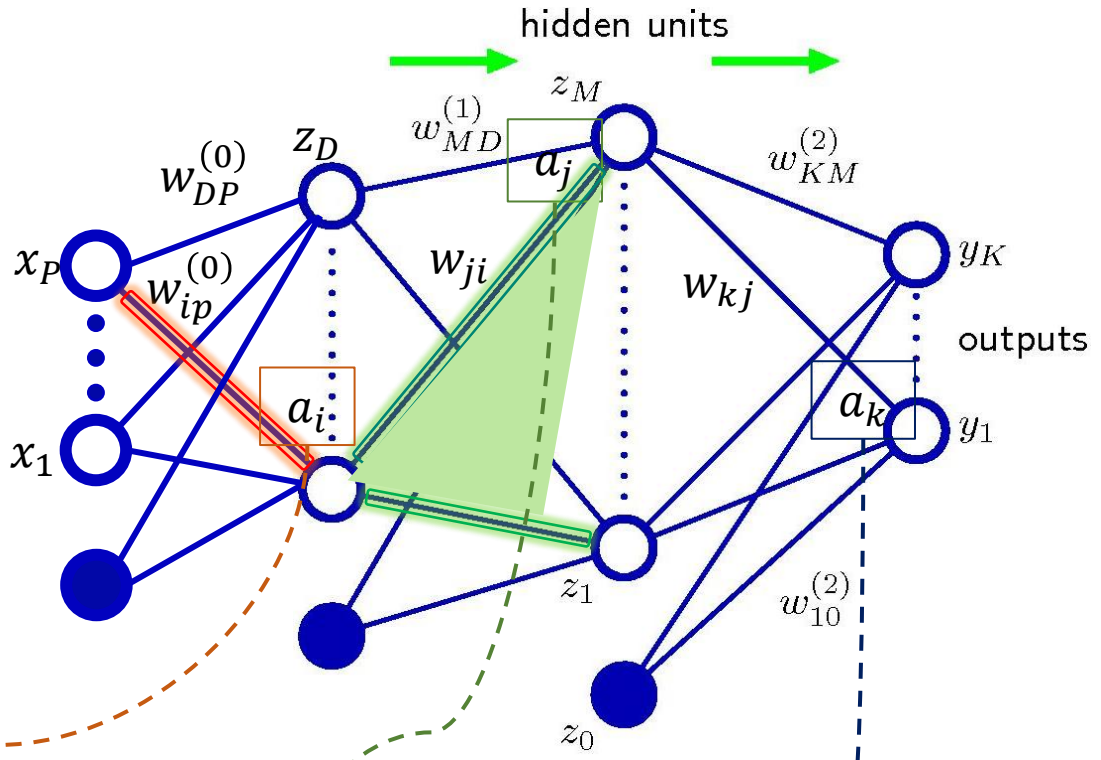
$$y_k = softmax(a_k)$$

$$a_k = \sum_{j=1}^{M} w_{kj}^{(2)} h(a_j)$$

$$a_j = \sum_{i=1}^{D} w_{ji}^{(1)} h(a_i)$$

$$a_i = \sum_{p=1}^{P} w_{ip}^{(0)} x_p$$

$$y_k = softmax\left(\sum_{j=1}^{M} w_{kj}^{(2)}\, h\left(\sum_{i=1}^{D} w_{ji}^{(1)} h\left(\sum_{p=1}^{P} w_{ip}^{(1)} x_p\right)\right)\right)$$



误差后向传播 Backward

$$\delta_i = \frac{\partial L}{\partial a_i} = h'(a_i) \sum_{j=1}^{M} w_{ji}^{(1)} \delta_j$$

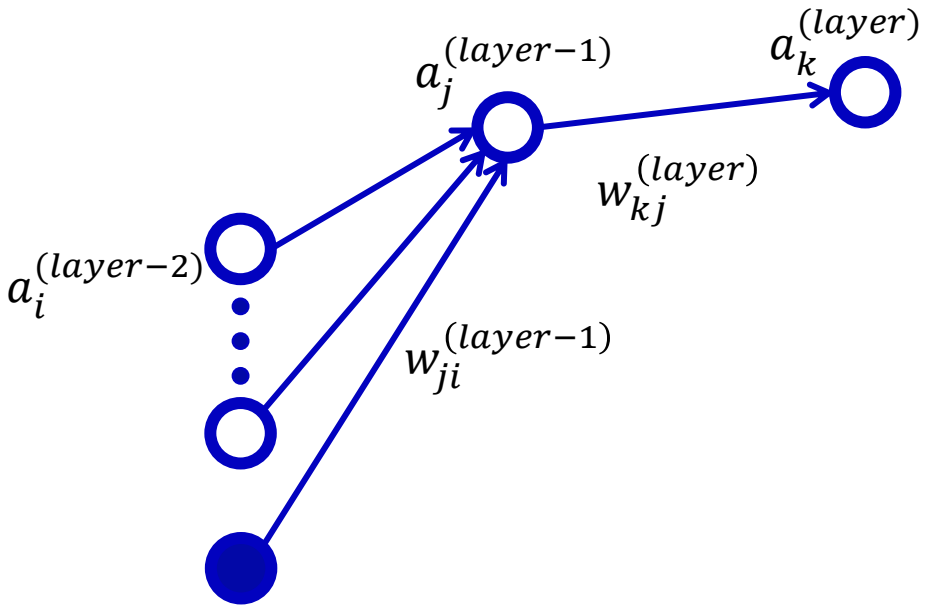
$$\delta_j = \frac{\partial L}{\partial a_j} = h'(a_j) \sum_{k=1}^{K} w_{kj}^{(2)} \delta_k$$

$$\delta_k = \frac{\partial L}{\partial a_k} = y_k - t_k$$

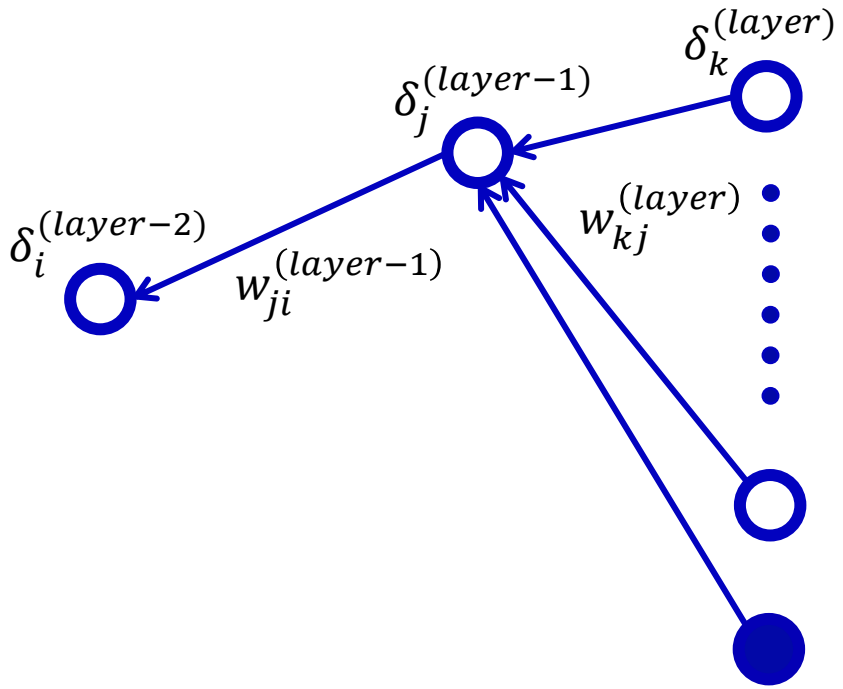
Δw_{ip} Δw_{ji} Δw_{kj}

4 误差后向传播：流程图

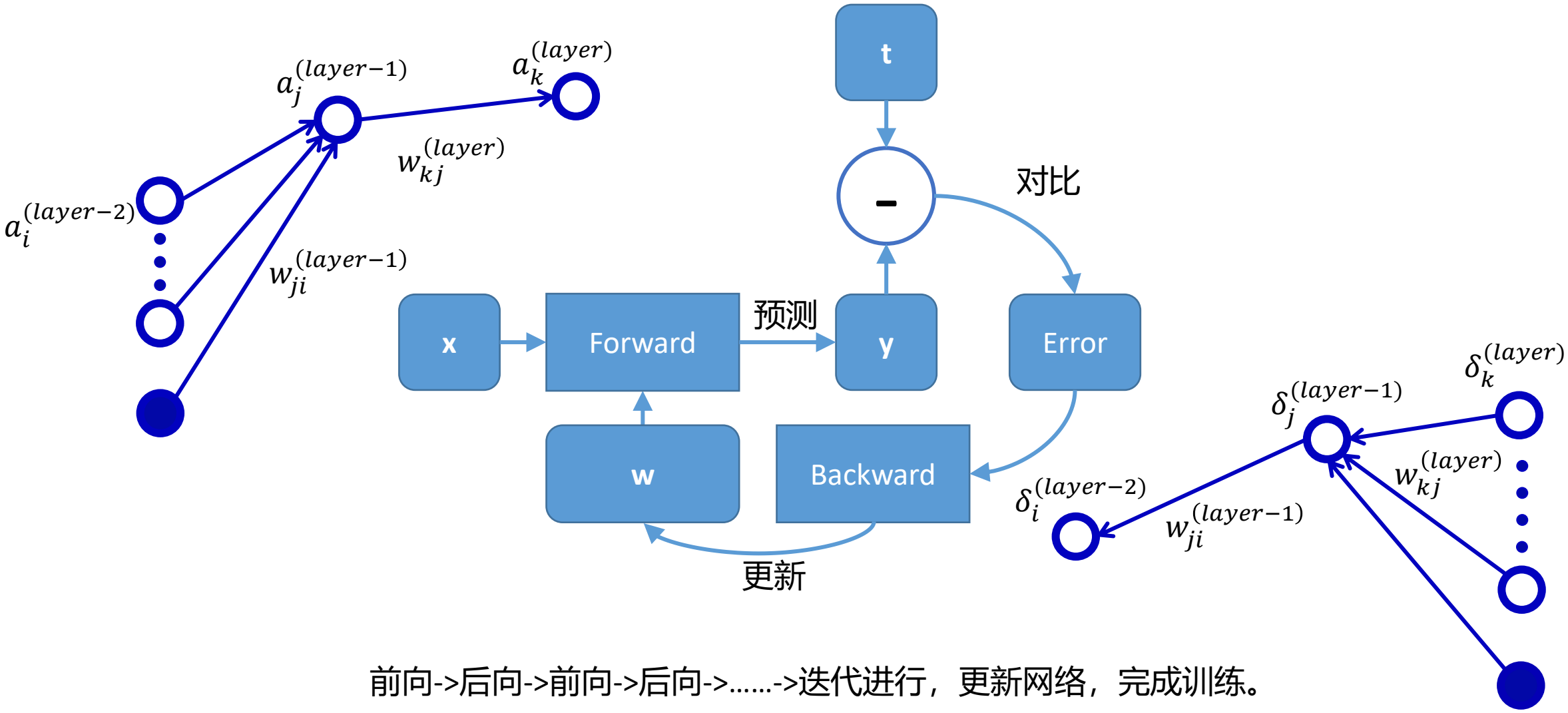
数据前向计算
Forward



误差后向传播
Backward



4 误差后向传播：流程图



前向->后向->前向->后向->……->迭代进行，更新网络，完成训练。

4 误差后向传播：数据前向计算矩阵形式

- 在实际计算中，矩阵形式可以更好的进行加速实现

$$y_k = softmax(a_k)$$

$$a_k = \sum_{j=1}^{M} w_{kj}^{(2)} h(a_j)$$

$$a_j = \sum_{i=1}^{D} w_{ji}^{(1)} h(a_i)$$

$$a_i = \sum_{p=1}^{P} w_{ip}^{(0)} x_p$$

$$\mathbf{a}^{(2)} = \begin{pmatrix} a_1^{(2)} \\ a_2^{(2)} \\ \vdots \\ a_K^{(2)} \end{pmatrix}$$

$$\mathbf{w}_{kj}^{(2)} = \begin{pmatrix} w_{k1}^{(2)} \\ w_{k2}^{(2)} \\ \vdots \\ w_{kM}^{(2)} \end{pmatrix}$$

$M \times 1$

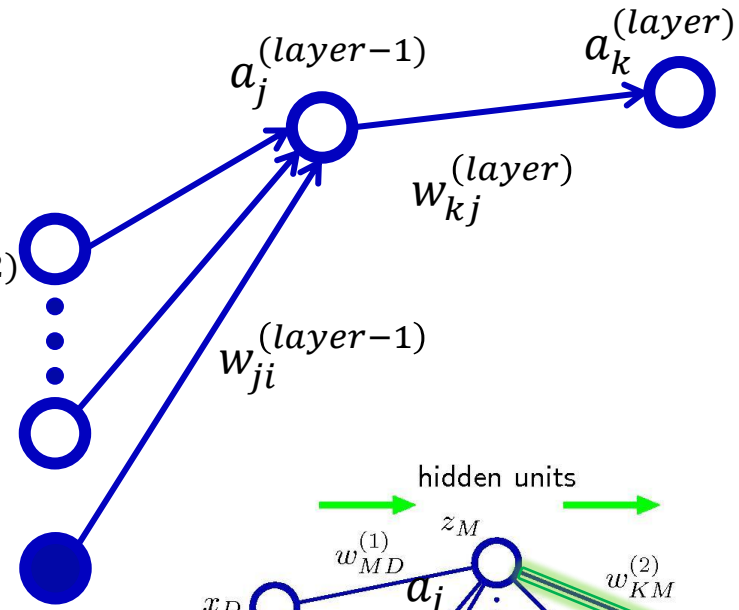
$$\mathbf{W}_{kj}^{(2)} = \begin{pmatrix} \mathbf{w}_{1j}^{(2)T} \\ \mathbf{w}_{2j}^{(2)T} \\ \vdots \\ \mathbf{w}_{Kj}^{(2)T} \end{pmatrix}$$

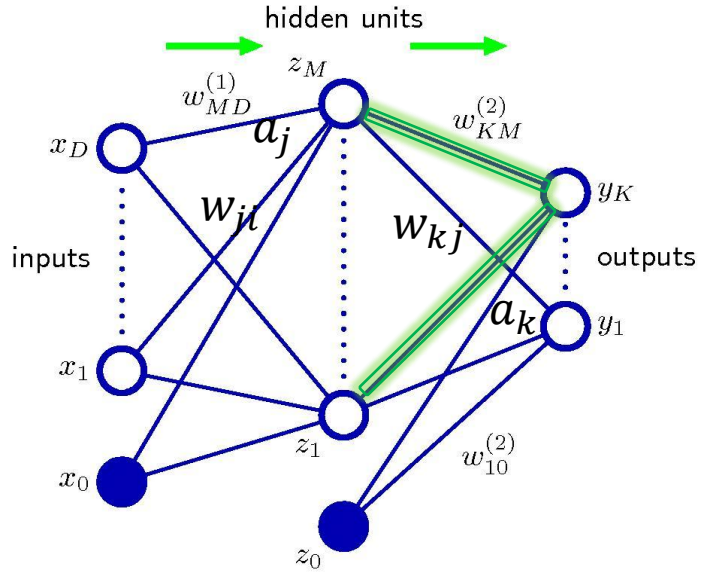
$K \times M$

$$\mathbf{z}^{(1)} = \begin{pmatrix} z_1^{(1)} \\ z_2^{(1)} \\ \vdots \\ z_M^{(1)} \end{pmatrix} = \begin{pmatrix} h\left(a_1^{(1)}\right) \\ h\left(a_2^{(1)}\right) \\ \vdots \\ h\left(a_M^{(1)}\right) \end{pmatrix}$$

$$\boxed{\mathbf{a}^{(2)} = \mathbf{W}_{kj}^{(2)} \mathbf{z}^{(1)}}$$

Forward





4 误差后向传播：误差后向传播矩阵形式

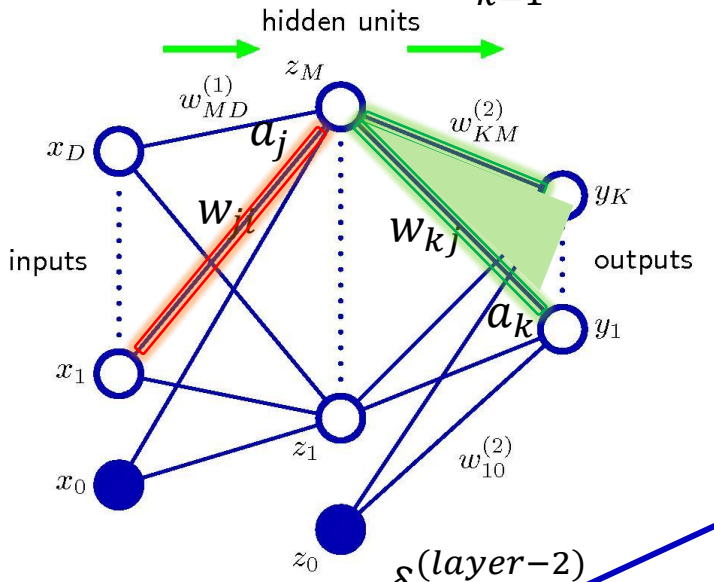
$$\boldsymbol{\delta}^{(1)} = \begin{pmatrix} \frac{\partial L}{\partial a_1^{(1)}} \\ \frac{\partial L}{\partial a_2^{(1)}} \\ \vdots \\ \frac{\partial L}{\partial a_M^{(1)}} \end{pmatrix} = \begin{pmatrix} \delta_1^{(1)} \\ \delta_2^{(1)} \\ \vdots \\ \delta_M^{(1)} \end{pmatrix} \quad \boldsymbol{\delta}^{(2)} = \begin{pmatrix} \frac{\partial L}{\partial a_1^{(2)}} \\ \frac{\partial L}{\partial a_2^{(2)}} \\ \vdots \\ \frac{\partial L}{\partial a_K^{(2)}} \end{pmatrix} = \begin{pmatrix} \delta_1^{(2)} \\ \delta_2^{(2)} \\ \vdots \\ \delta_K^{(2)} \end{pmatrix}$$

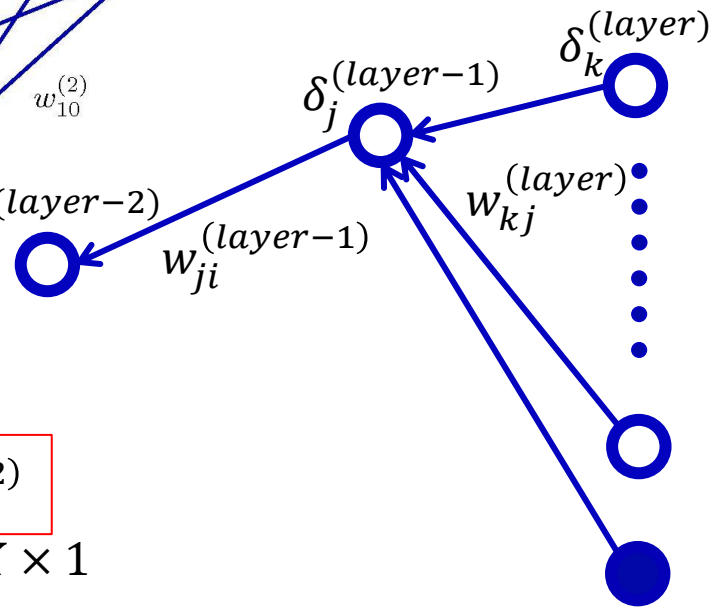
$$\mathbf{w}_{jk}^{(2)} = \begin{pmatrix} w_{j1}^{(2)} \\ w_{j2}^{(2)} \\ \vdots \\ w_{jK}^{(2)} \end{pmatrix} \quad \mathbf{W}_{jk}^{(2)} = \begin{pmatrix} \mathbf{w}_{1k}^{(2)T} \\ \mathbf{w}_{2k}^{(2)T} \\ \vdots \\ \mathbf{w}_{Mk}^{(2)T} \end{pmatrix} \quad \mathbf{h}'^{(1)} = \begin{pmatrix} h'\left(a_1^{(1)}\right) \\ h'\left(a_2^{(1)}\right) \\ \vdots \\ h'\left(a_M^{(1)}\right) \end{pmatrix}$$

$K \times 1$ $\quad M \times K$ $\quad M \times 1$

Backward

$$\delta_j^{(layer-1)} = h'\left(a_j^{(layer-1)}\right) \sum_{k=1}^{K} w_{kj}^{(layer)} \delta_k^{(layer)}$$





$$\boldsymbol{\delta}^{(1)} = \left(\mathbf{h}'^{(1)}\right) \cdot \mathbf{W}_{jk}^{(2)} \boldsymbol{\delta}^{(2)}$$

$M \times 1$ $\quad M \times 1$ $M \times K$ $K \times 1$

4 误差后向传播：矩阵形式

逐层前向

$$\mathbf{w}_{kj}^{(2)} = \begin{pmatrix} w_{k1}^{(2)} \\ w_{k2}^{(2)} \\ \vdots \\ w_{kM}^{(2)} \end{pmatrix} \quad \mathbf{W}_{kj}^{(2)} = \begin{pmatrix} \mathbf{w}_{1j}^{(2)T} \\ \mathbf{w}_{2j}^{(2)T} \\ \vdots \\ \mathbf{w}_{Kj}^{(2)T} \end{pmatrix} = \begin{pmatrix} w_{11}^{(2)} w_{12}^{(2)} \cdots w_{1M}^{(2)} \\ w_{21}^{(2)} w_{22}^{(2)} \cdots w_{2M}^{(2)} \\ \vdots \quad \vdots \quad \ddots \quad \vdots \\ w_{K1}^{(2)} w_{K2}^{(2)} \cdots w_{KM}^{(2)} \end{pmatrix}$$

$M \times 1$ $K \times M$

计算线性组合和激活函数输出

$$\mathbf{a}^{(L)} = \mathbf{W}_{kj}^{(L)} \mathbf{z}^{(L-1)}$$

$$\mathbf{z}^{(L-1)} = \mathbf{h}\left(\mathbf{a}^{(L-1)}\right)$$

逐层后向

$$\mathbf{w}_{jk}^{(2)} = \begin{pmatrix} w_{j1}^{(2)} \\ w_{j2}^{(2)} \\ \vdots \\ w_{jK}^{(2)} \end{pmatrix} \quad \mathbf{W}_{jk}^{(2)} = \begin{pmatrix} \mathbf{w}_{1k}^{(2)T} \\ \mathbf{w}_{2k}^{(2)T} \\ \vdots \\ \mathbf{w}_{Mk}^{(2)T} \end{pmatrix} \quad \mathbf{W}_{jk}^{(2)T} = \begin{pmatrix} w_{11}^{(2)} w_{12}^{(2)} \cdots w_{1K}^{(2)} \\ w_{21}^{(2)} w_{22}^{(2)} \cdots w_{2K}^{(2)} \\ \vdots \quad \vdots \quad \ddots \quad \vdots \\ w_{M1}^{(2)} w_{M2}^{(2)} \cdots w_{MK}^{(2)} \end{pmatrix}$$

$K \times 1$ $M \times K$ $M \times K$

先传播所有误差

$$\boldsymbol{\delta}^{(L-1)} = \left(\mathbf{h}'^{(L-1)}\right) \cdot \mathbf{W}_{jk}^{(L)} \boldsymbol{\delta}^{(L)}$$

$M \times 1$ $M \times 1$ $M \times K$ $K \times 1$

再更新权值

$$\nabla \mathbf{W}_{ji}^{(L)} = \boldsymbol{\delta}^{(L)} \left(\mathbf{z}^{(L-1)}\right)^T$$

$M \times D$ $M \times 1$ $1 \times D$

$$\mathbf{z}^{(0)} = \mathbf{x}$$

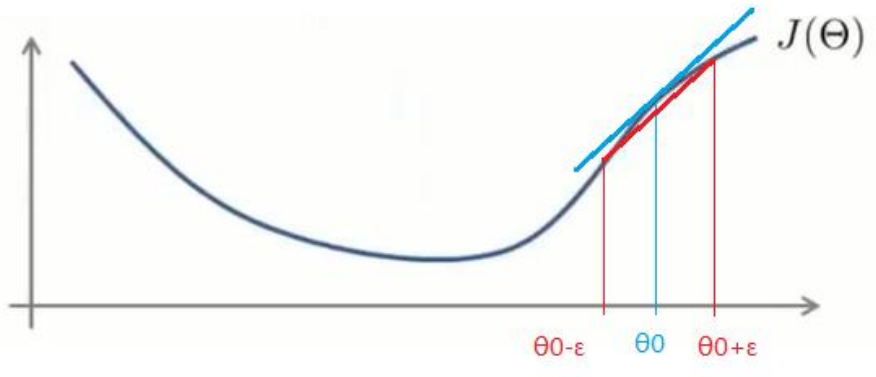
$$\frac{\partial L}{\partial w_{ji}^{(1)}} = \delta_j z_i$$

$$\mathbf{W}_{jk}^{(2)T} = \mathbf{W}_{kj}^{(2)} \qquad w_{1K}^{(2)} = w_{K1}^{(2)}$$

不同的下标顺序，表示通过该权值的数据流动方向不同，但权值共享同一个。

4 误差后向传播：梯度检测

1. 在后向传播的编程实现中，由于函数复杂度高，很容易出现梯度计算方面的微小错误。
2. 这些Bug不会影响运行，并且在迭代中，损失函数也在变小。
3. 但最终结果会远比无Bug的结果差（数量级）
4. 因此在计算梯度时，要以数值计算方式进行检验，梯度检测。
5. 注意：梯度检测的计算量非常大，一旦检测确认计算无误，应立即关闭梯度检测代码

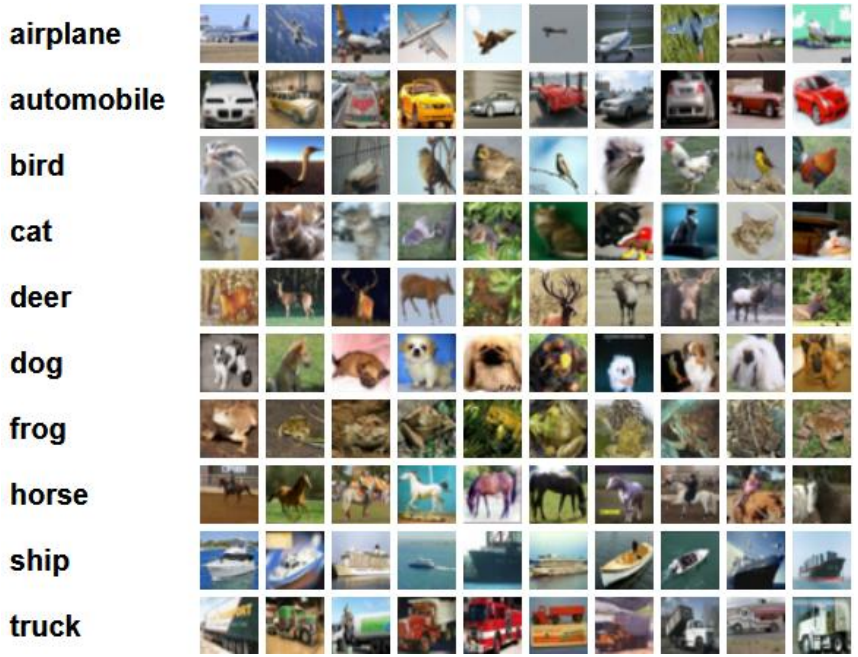


一般来说ε的值选10^{-4}，不是越小越好

作业

MLP神经网络完成下列任务。要使用验证集，搜索最优超参数；暂不考虑正则化或只使用权值衰减策略。

- CIFAR-10，图像分类，特征为所有像素按顺序排列构成的向量， Pytorch



感谢！



4 误差后向传播：计算复杂度

- 每个权重对应一次加法和一次乘法
- 与权重相比，节点上的激活操作占比很小，可忽略
- 总体算法复杂度 $O(W)$, W为权值参数总数
- 后向传播与前向计算的复杂度相同